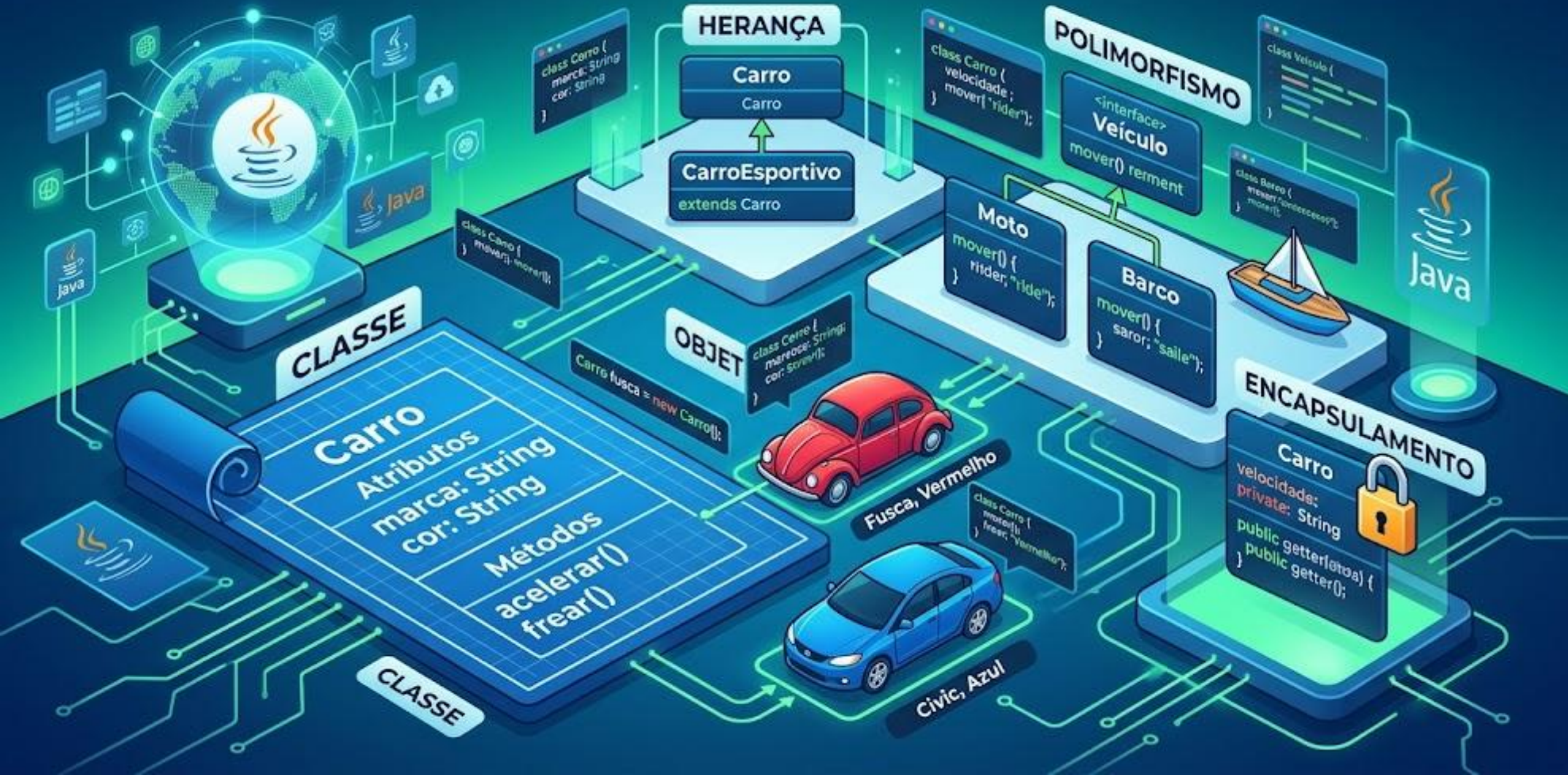


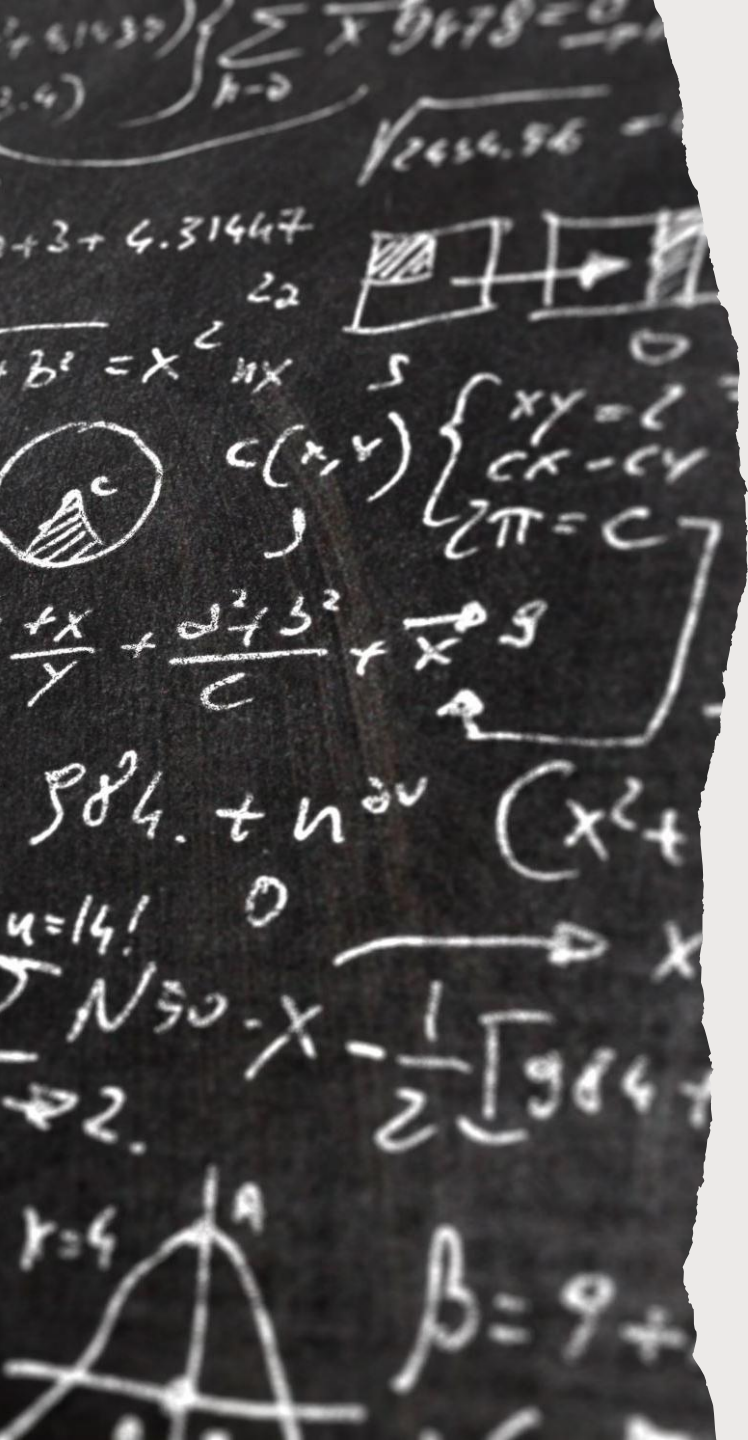


Developer Academy

www.developeracademy.com.br

ORIENTAÇÃO A OBJETOS EM JAVA





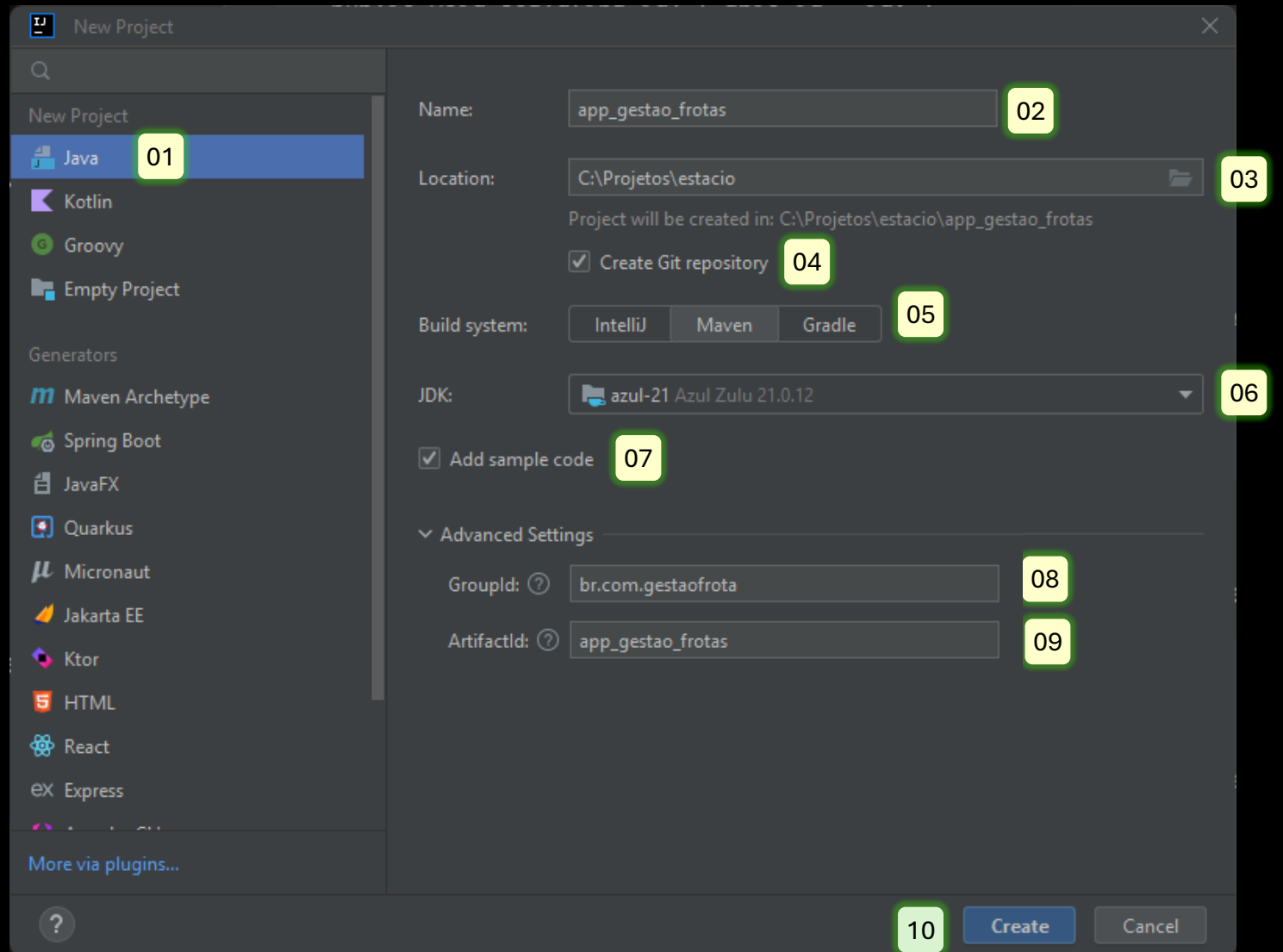
Programação Orientada a Objetos

Prof. Me. Fabrício Leonard Leopoldino

Guia de Desenvolvimento ao Vivo

Projeto 01 - Gestão de Frota

1. Configuração Inicial do Projeto no IntelliJ IDEA



app_gestao_frotas - pom.xml (app_gestao_frotas)

Project

- app_gestao_frotas C:\Projetos\estacio\app_gestao_frotas
 - .idea
 - .mvn
 - src
 - main
 - java
 - br.com.gestaofrota
 - Main
 - resources
 - test
 - target
 - .gitignore
 - m pom.xml
- External Libraries
- Scratches and Consoles

01

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion> Compatible with Maven 3
5
6     <groupId>br.com.gestaofrota</groupId>
7     <artifactId>app_gestao_frotas</artifactId>
8     <version>1.0-SNAPSHOT</version>
9
10    <properties>
11        <maven.compiler.source>21</maven.compiler.source>
12        <maven.compiler.target>21</maven.compiler.target>
13        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
14    </properties>
15
16 </project>
```

02

03

15:1 LF UTF-8 4 spaces master

FileEditViewNavigateCodeRefactorBuildRunToolsGitWindowHelpapp_gestao_frotas - Main.java

app_gestao_frotas src main java br com gestaofrotaMain03

Project
app_gestao_frotas C:\Projetos\estacio\app_gestao_frota
 > .idea
 > .mvn
 > src
 > main
 > java
 > br.com.gestaofrota
 Main
 resources
 test
 target
 .gitignore
 pom.xml
 > External Libraries
 > Scratches and Consoles

01

02

03

Current File

Git

1package br.com.gestaofrota;
2
3//TIP To Run code, press <shortcut actionId="Run"/> or
4// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
5public class Main {
6 public static void main(String[] args) {
7 //TIP Press <shortcut actionId="ShowIntentionActions"/> with your caret at the highlighted text
8 // to see how IntelliJ IDEA suggests fixing it.
9 System.out.printf("Hello and welcome!");
10
11 for (int i = 1; i ≤ 5; i++) {
12 //TIP Press <shortcut actionId="Debug"/> to start debugging your code. We have set one <icon src="AllIcons.D
13 // for you, but you can always add more by pressing <shortcut actionId="ToggleLineBreakpoint"/>.
14 System.out.println("i = " + i);
15 }
16 }
17}

Notifications
AI Chat
Endpoints
Database
Maven
Coverage

GitRunProblemsTerminalTODOProfilerServicesBuild

17:2 L F UTF-8 4 spaces master

FileEditViewNavigateCodeRefactorBuildRunToolsGitWindowHelpapp_gestao_frotas - Main.java

app_gestao_frotas src main java br com gestaofrotaMain

Project

app_gestao_frotas C:\Projetos\estacio\app_gestao_frota

main

java

br.com.gestaofrota

resources

test

target

.gitignore

pom.xml

External Libraries

Scratches and Consoles

5

6

7

10

11

12

13

14

15

16

17

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello and welcome!");  
  
        for (int i = 1; i <= 5; i++) {  
            //TIP Press <shortcut actionId="Debug"/> to start debugging your code. We have set one <icon src="AllIcons.D  
            // for you, but you can always add more by pressing <shortcut actionId="ToggleLineBreakpoint"/>.  
            System.out.println("i = " + i);  
        }  
    }  
}
```

01

Run: Main

Run

C:\Users\fabricaoleonard\.jdk\azul-21.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.2.3\lib\idea_rt.jar=59993"
-Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Projetos\estacio\app_gestao_frotas\target\classes
br.com.gestaofrota.Main
Hello and welcome!
i = 1
i = 2
i = 3
i = 4
i = 5

Process finished with exit code 0

02

GitRunProblemsTerminalTODOProfilerServicesBuild

// Build completed successfully in 2 sec, 317 ms (moments ago)

17:2 L F UTF-8 4 spaces master

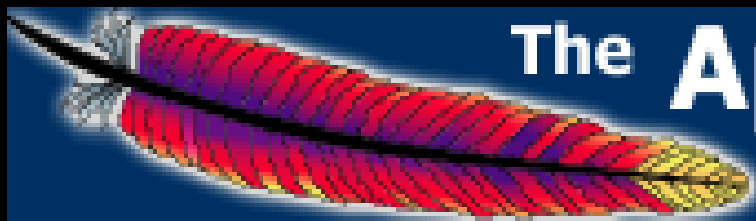


O **Apache Maven** é uma das ferramentas de gerenciamento e automação de compilação (*build*) mais populares do ecossistema Java. Desenvolvido pela Apache Software Foundation (<https://maven.apache.org/download.cgi>), ele foi projetado para simplificar a criação, a padronização e a manutenção de projetos de software. Em termos simples, o Maven cuida do "trabalho pesado" de infraestrutura do seu projeto: baixa bibliotecas necessárias, compila os arquivos .java, executa testes automatizados, empacota o código em arquivos executáveis (como .jar ou .war) e os prepara para implantação.



O Arquivo pom.xml (Project Object Model)

- O coração de qualquer projeto Maven é o arquivo **pom.xml**, localizado na raiz do projeto. Ele funciona como o "manual de instruções" ou a receita do seu projeto, contendo:
- **Coordenadas do projeto:** Identificadores únicos (groupId, artifactId e version).
- **Dependências:** A lista de bibliotecas externas de que o código precisa para rodar (ex: Spring Boot, JUnit, Hibernate).
- **Plugins:** Extensões que adicionam comportamentos customizados ao processo de build.
- **Propriedades:** Configurações como a versão do Java utilizada.



The Apache Maven Project

<http://maven.apache.org/>

Project ▾ + ⊕ ≡ ⚙ -

app_gestao_frotas C:\Projetos\estacio\app_gestao_frota

- .idea
- .mvn
- ▾ src
 - ▾ main
 - ▾ java
 - ▾ br.com.gestaofrota
 - Main
 - resources
 - test
 - target
 - .gitignore
 - m pom.xml
- External Libraries
- Scratches and Consoles

Main.java x m pom.xml (app_gestao_frotas) x

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion> Compatible with Maven 3
7
8     <groupId>br.com.gestaofrota</groupId>
9     <artifactId>app_gestao_frotas</artifactId>
10    <version>1.0-SNAPSHOT</version>
11
12    <properties>
13        <maven.compiler.source>21</maven.compiler.source>
14        <maven.compiler.target>21</maven.compiler.target>
15        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16    </properties>
17
18 </project>
19 |
```

2. Adicionar a dependência do driver JDBC do SQLite

```
<dependencies>  
  <!-- Driver JDBC do SQLite -->  
  <dependency>  
    <groupId>org.xerial</groupId>  
    <artifactId>sqlite-jdbc</artifactId>  
    <version>3.45.1.0</version>  
  </dependency>  
</dependencies>
```




SQLite: Banco de Dados Embutido e Leve (<https://sqlite.org/>)

- **Arquitetura Serverless:** Não requer instalação ou gerenciamento de um servidor de banco de dados separado.
- **Arquivo Único:** Armazena todas as tabelas, índices e dados do projeto em um único arquivo .db.
- **Zero Configuração:** Pronto para uso imediato, sem necessidade de criação de usuários ou gestão de portas.
- **Leve e Confiável:** Suporte completo a transações ACID com consumo mínimo de memória e processamento.
- **Casos de Uso:** Ideal para apps mobile (iOS/Android), software desktop, dispositivos IoT e prototipagem rápida.

FileEditViewNavigateCodeRefactorBuildRunToolsGitWindowHelp

app_gestao_frotas - pom.xml (app_gestao_frotas)

app_gestao_frotas pom.xml

Project
app_gestao_frotas
 .idea
 .mvn
 src
 main
 java
 br.com.gestaofrota
 Main
 resources
 test
 target
 .gitignore
 pom.xml
 External Libraries
 Scratches and Consoles

02

01

24:1 LF UTF-8 4 spaces master

Run: Main

GitRunProblemsTerminalTODOProfilerServicesBuild

// Build completed successfully in 2 sec, 317 ms (2 minutes ago)

25

<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion> Compatible with Maven 3

<groupId>br.com.gestaofrota</groupId>

<artifactId>app_gestao_frotas</artifactId>

<version>1.0-SNAPSHOT</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

</properties>

<dependencies>

<!-- Driver JDBC do SQLite -->

<dependency>

<groupId>org.xerial</groupId>

<artifactId>sqlite-jdbc</artifactId>

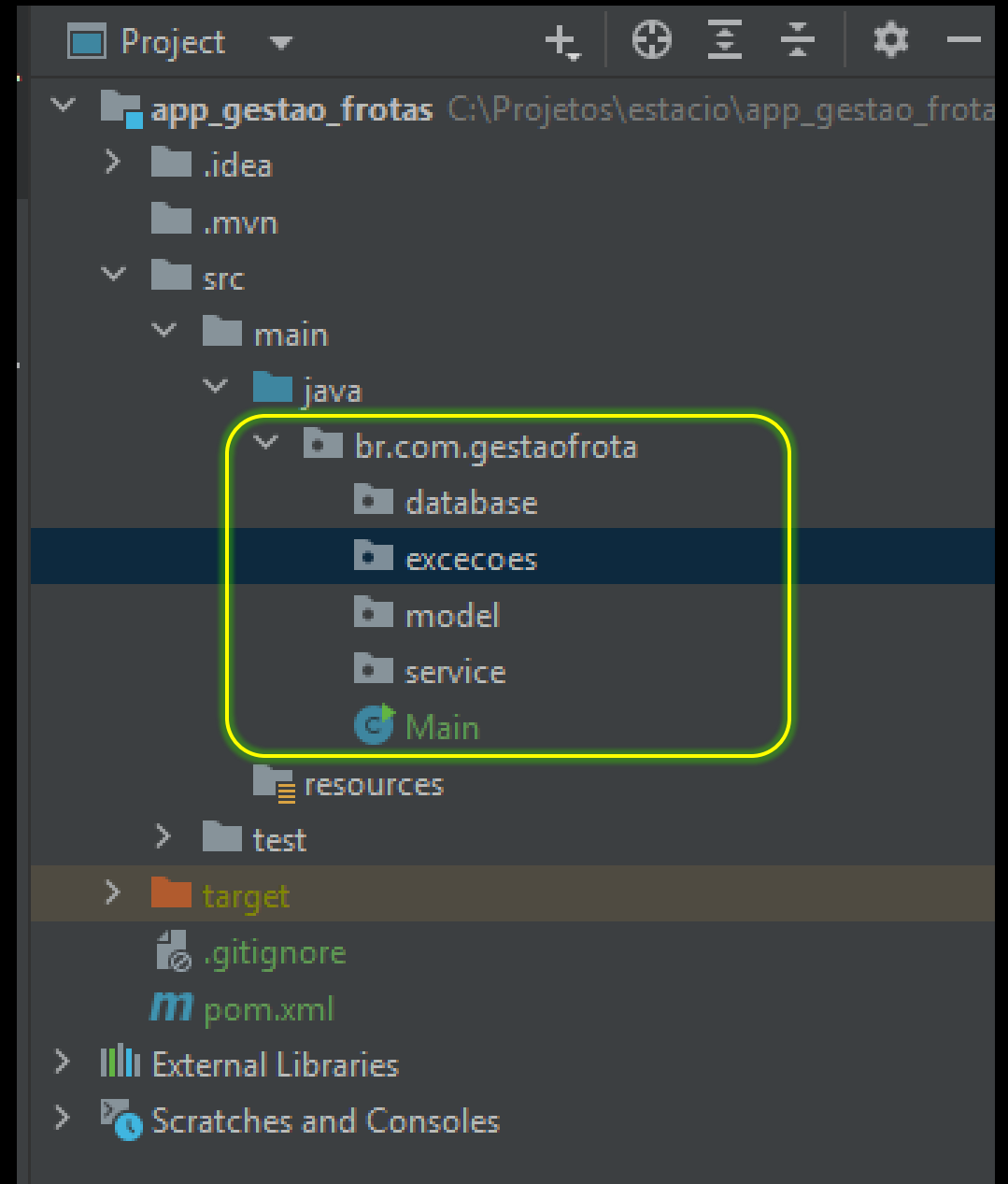
<version>3.45.1.0</version>

</dependency>

</dependencies>

</project>

3. Estrutura de Pacotes do Projeto



Pacotes (Packages) em Java



Organização: Agrupam classes e interfaces relacionadas em pastas lógicas no projeto.



Evitam Conflitos de Nomes: Permitem a coexistência de classes com nomes iguais em pacotes diferentes (ex: `java.util.Date` vs `java.sql.Date`).



Controle de Acesso: Ajudam a proteger o código definindo visibilidade (*package-private* e *protected*).

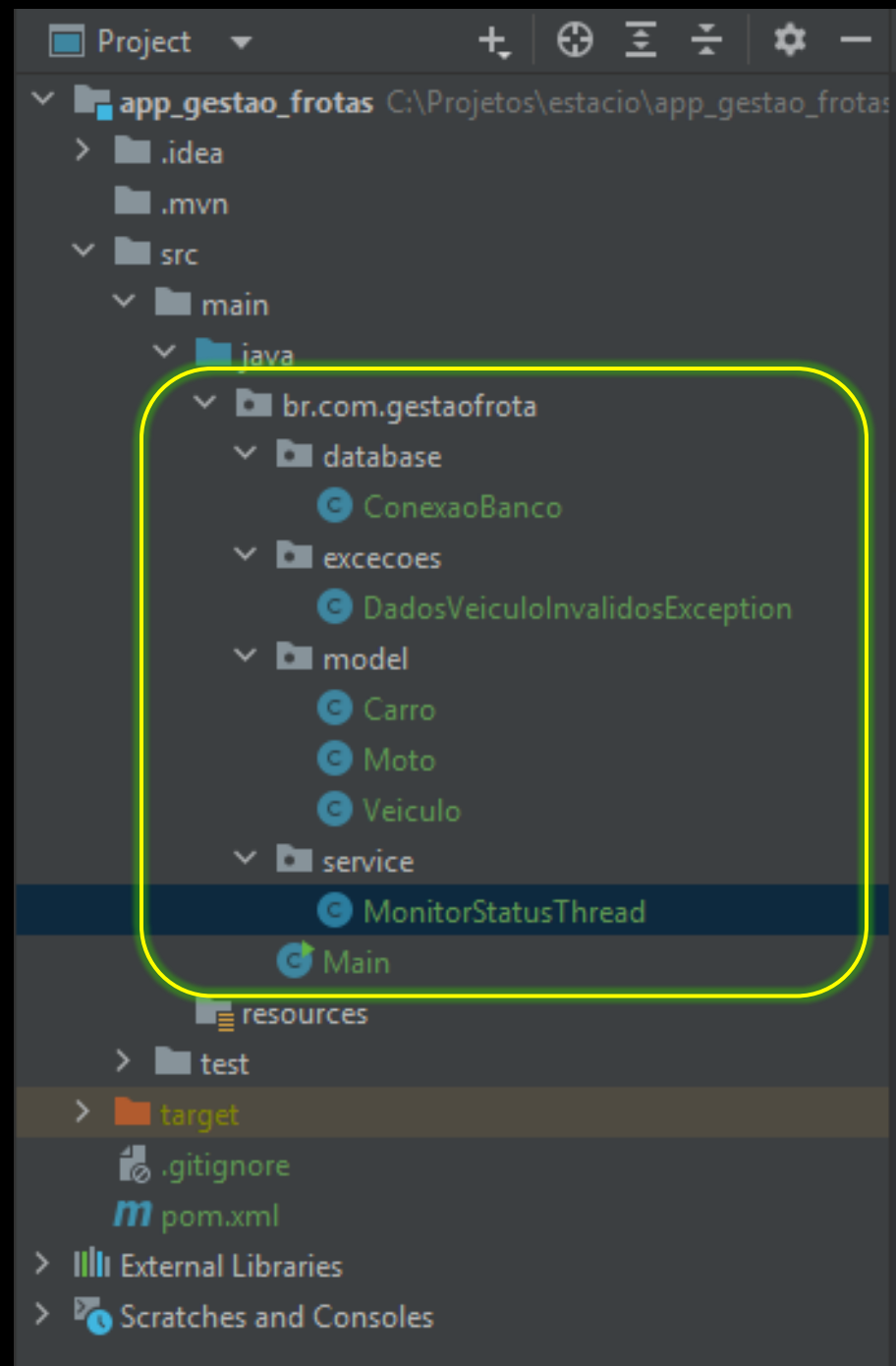


Reutilização: Facilitam a distribuição e importação de módulos e bibliotecas no Maven.



Convenção de Nome: Usa o domínio invertido da organização: `br.com.empresa.projeto.camada`

4. Adicione os arquivos com a extensão “.java”, obedecendo a regra CamelCase



Convenção CamelCase em Java

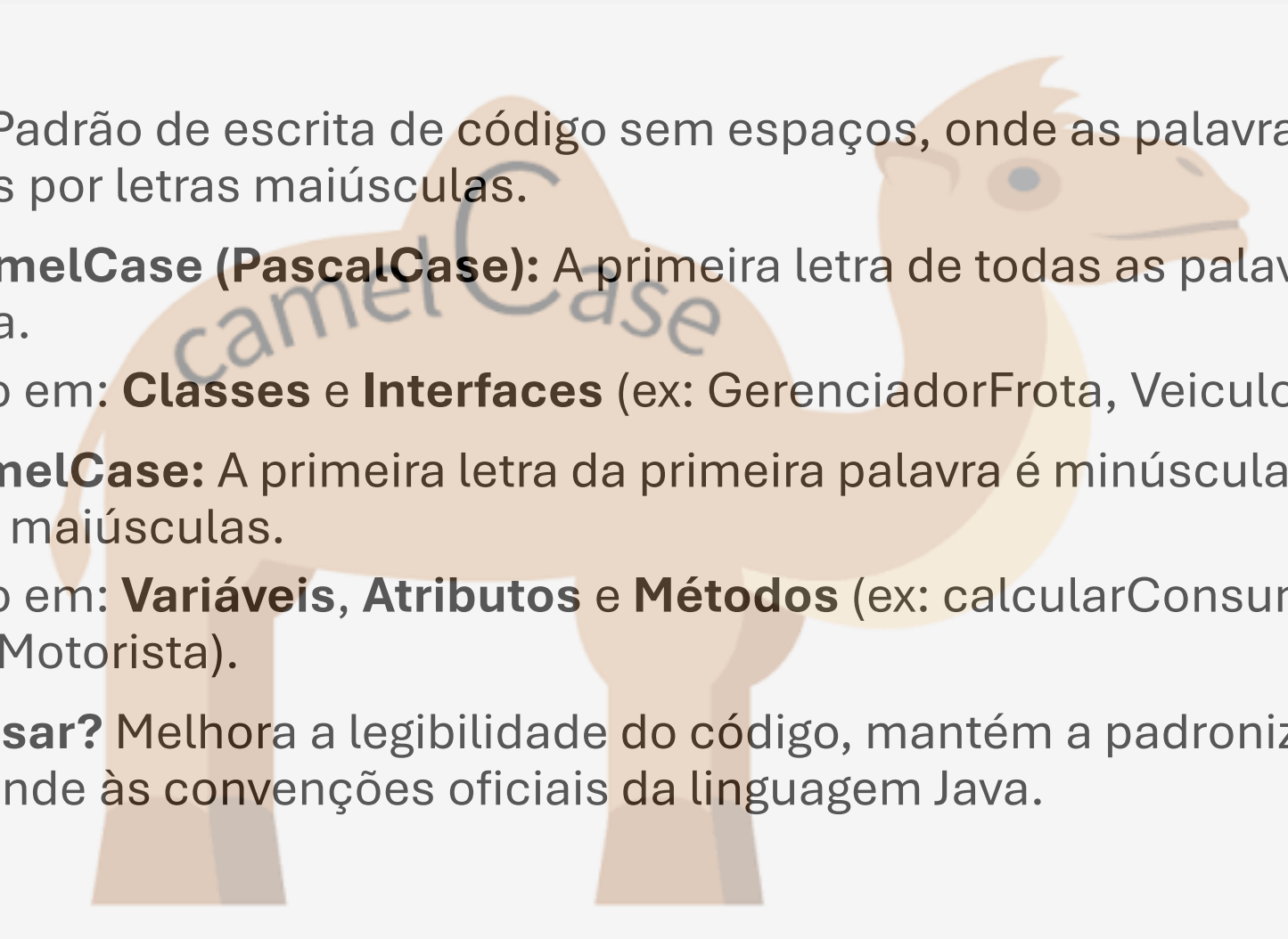
- 
- **O que é:** Padrão de escrita de código sem espaços, onde as palavras são separadas por letras maiúsculas.
 - **UpperCamelCase (PascalCase):** A primeira letra de todas as palavras é maiúscula.
 - Usado em: **Classes** e **Interfaces** (ex: GerenciadorFrota, VeiculoService).
 - **lowerCamelCase:** A primeira letra da primeira palavra é minúscula e as seguintes maiúsculas.
 - Usado em: **Variáveis**, **Atributos** e **Métodos** (ex: calcularConsumo(), nomeMotorista).
 - **Por que usar?** Melhora a legibilidade do código, mantém a padronização no time e atende às convenções oficiais da linguagem Java.

Tabela Comparativa

Elemento	Padrão CamelCase	Exemplo
Classes / Interfaces	UpperCamelCase	ContaBancaria, RelatorioService
Métodos	lowerCamelCase	obterSaldo(), adicionarVeiculo()
Variáveis / Atributos	lowerCamelCase	dataNascimento, limiteCredito
Pacotes	Tudo minúsculo	br.com.empresa.projeto
Constantes	SNAKE_CASE (Maiúsculas)	MAX_TENTATIVAS, TAXA_PADRAO



5. Desenvolvimento Passo a Passo

An abstract geometric pattern on the left side of the slide. It features a grid of thin grey lines with small grey circles at the intersections. Overlaid on this grid are various squares of different colors (black, green, blue, yellow, red, purple, cyan) and sizes. Some squares are solid, while others are outlined. Some squares have a slight 3D shadow effect. The pattern is more dense on the left and fades out towards the right.

Blocos

- Orientação a Objetos, Herança e Polimorfismo
- Exceções Customizadas
- Integração com Banco de Dados (JDBC + SQLite)
- Programação Paralela com Threads
- Classe Principal e Execução

Bloco 1:

Orientação a Objetos, Herança e Polimorfismo

Concept Highlight

- **Atributos privados e métodos acessores:** Encapsulamento de estado.
- **Herança (extends):** Reuso do modelo **Veiculo** para especificação em **Carro** e **Moto**.
- **Polimorfismo (@Override):** Sobrescrita do método `exibirFichaTecnica()` para compor o comportamento específico de cada subclasse.

Project

app_gestao_frotas

C:\Projetos\estacio\app_gestao_frotas

.idea

.mvn

src

main

java

br.com.gestaofrota

database

ConexaoBanco

excecoes

DadosVeiculoInvalidosException

model

Carro

Moto

Veiculo

service

MonitorStatusThread

Main

resources

test

target

.gitignore

pom.xml

External Libraries

Scratches and Consoles

Main.java

pom.xml (app_gestao_frotas)

ConexaoBanco.java

DadosVeiculo

1

2

3

4

5

```
package br.com.gestaofrota.model;

public class Veiculo {
}
```

O Papel da Classe no Paradigma Orientado a Objetos

Uma **Classe** em Java é a estrutura primária que encapsula dados e comportamentos em uma única unidade. Ela abstrai conceitos do domínio do sistema em código executável: enquanto a classe representa a definição abstrata de uma regra ou entidade, o **objeto** representa a instância concreta dessa classe alocada na memória durante a execução do programa.

CLASSES E OBJETOS NO JAVA

CONCEITO E EXEMPLO PRÁTICO

A CLASSE (O MOLDE)



CLASSE: Veiculo

- Molde abstrato
- Define **Atributos** (estado): modelo, placa, cor, km
- Define **Métodos** (comportamento): ligar(), acelerar(), frear()

OS OBJETOS (INSTÂNCIAS)

OBJETOS: Instâncias Concretas

new Veiculo(...)

veiculo1

Atributos:
modelo='Corolla',
placa='ABC-1234',
cor='Vermelho'

veiculo2

Atributos:
modelo='Corolla',
placa='ABC-1234',
cor='Vermelho'

veiculo2

Atributos:
modelo='Civic',
placa='XYZ-9876',
cor='Azul'

veiculo3

Atributos:
modelo='GOL',
placa='DEF-4567',
cor='Amarelo'



Definição de Atributos

Os **atributos** são as variáveis de uma classe responsável por armazenar as informações e propriedades que descrevem uma entidade no sistema. Enquanto os *métodos* definem **o que o objeto faz**, os *atributos* definem **como o objeto é**. Em Java, a alteração e leitura desses valores ocorrem sob o princípio do encapsulamento, garantindo o controle sobre a integridade dos dados alocados na memória.

```
Veiculo.java x
1 package br.com.gestaofrota.model;
2
3 public class Veiculo {
4
5     // 1 - Atributos
6     private Integer id;
7     private String marca;
8     private String modelo;
9     private int ano;
10
11 }
```

O Papel do Construtor no Ciclo de Vida do Objeto

O **construtor** é o método responsável por garantir que um objeto nasça em um estado válido e pronto para uso. Ele define quais informações são obrigatórias na hora de instanciar a classe, atribuindo os valores iniciais aos seus atributos e alocando os recursos necessários na memória.

```
11 // 2 - Construtor
12 public Veiculo() {
13 }
14
15 public Veiculo(Integer id, String marca, String modelo, int ano) {
16     this.id = id;
17     this.marca = marca;
18     this.modelo = modelo;
19     this.ano = ano;
20 }
21
22 public Veiculo(String marca, String modelo, int ano) {
23     this.marca = marca;
24     this.modelo = modelo;
25     this.ano = ano;
26 }
```

O Papel dos Métodos na Orientação a Objetos

Os **Métodos** representam o comportamento dos objetos na linguagem Java. Enquanto os atributos armazenam o *estado* (dados) do objeto, os métodos encapsulam a *lógica* para manipular esses dados ou interagir com outros componentes do sistema, garantindo um código modular, limpo e testável.

O Papel dos Métodos na Orientação a Objetos

- **O que são:** Blocos de código reutilizáveis pertencentes a uma classe que realizam uma ação ou executam uma regra de negócio específica.
- **Objetivo Principal:** Promover o **reuso de código**, modularização e organização da lógica do sistema.
- **Anatomia de um Método:**
 - **Modificador de Acesso:** Define quem pode chamar o método (public, private, protected).
 - **Tipo de Retorno:** O tipo do dado devolvido pelo método (int, String, double, objetos ou void quando não retorna nada).
 - **Nome do Método:** Identificador escrito no padrão *camelCase* (ex: calcularConsumo).
 - **Parâmetros:** Dados de entrada necessários para a execução do método (opcional).
 - **Corpo do Método:** Bloco delimitado por chaves {} contendo a lógica e a instrução return (quando houver retorno).

Recursos Especiais de Métodos em Java



Sobrecarga de Métodos (*Overloading*): Criar múltiplos métodos na mesma classe com o **mesmo nome**, mas com **parâmetros diferentes** (tipos ou quantidade).



Sobrescrita de Métodos (*Overriding*): Redefinir em uma subclasse o comportamento de um método herdado da superclasse (utilizando a anotação `@Override`).



Métodos Estáticos (*static*): Métodos que pertencem à **classe** e não às instâncias de objetos (podem ser chamados diretamente como `Classe.metodo()`).

O Papel dos Modificadores no Encapsulamento

Os Modificadores de Acesso são a base do pilar de Encapsulamento na Orientação a Objetos em Java. Eles garantem a segurança do código ao restringir o acesso direto a métodos utilitários ou sensíveis (``private``), expondo para o restante da aplicação apenas a interface pública (``public``) necessária para a navegação e consumo dos serviços.

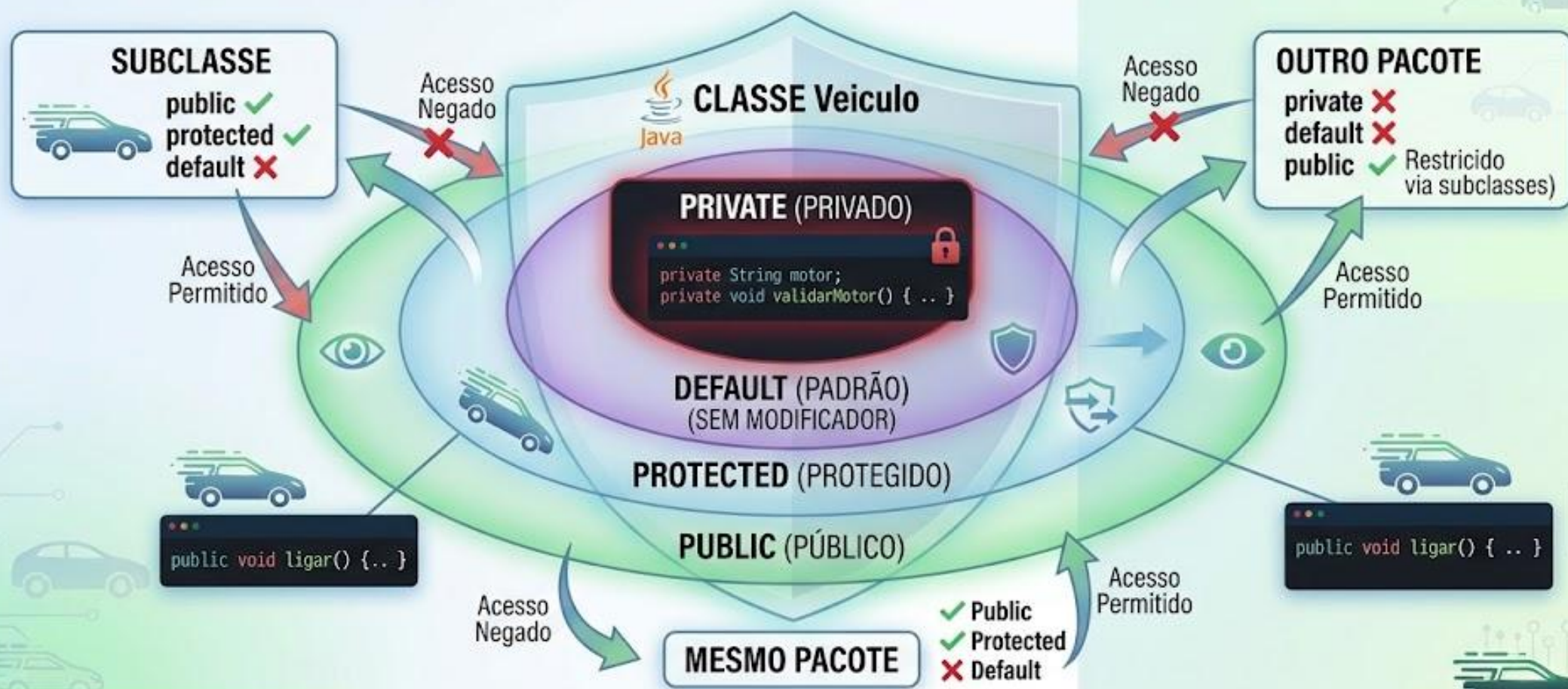
O Papel dos Modificadores no Encapsulamento

- **O que são:** Palavras-chave que determinam **quais outras classes podem acessar e invocar** um determinado método.
- **Os 4 Níveis de Visibilidade:**
 - **public (Público):** O método pode ser acessado de **qualquer lugar** da aplicação (qualquer classe, pacote ou subclasse).
 - **protected (Protegido):** O método é visível dentro do **mesmo pacote** e por **subclasses** em pacotes diferentes (herança).
 - **default / Package-Private (Padrão):** Ocorre quando nenhum modificador é informado. O método só é visível para classes do **mesmo pacote**.
 - **private (Privado):** O método é visível e acessível **exclusivamente dentro da própria classe** onde foi declarado.

Modificador	Própria Classe	Mesmo Pacote	Subclasse (Outro Pacote)	Todo o Sistema
public	Sim	Sim	Sim	Sim
protected	Sim	Sim	Sim	Não
default (sem palavra)	Sim	Sim	Não	Não
private	Sim	Não	Não	Não

ENCAPSULAMENTO: MÉTODOS E ESCOPO DE VISIBILIDADE

PROTEGER DADOS E CONTROLAR O ACESSO NO JAVA



```
28      // 3 - Método polimórfico
29      public String exibirFichaTecnica() {
30          return String.format("[%s] %s %s - Ano: %d",
31                                getClass().getSimpleName().toUpperCase(), marca, modelo, ano);
32      }
33
```

O Papel do Polimorfismo na Orientação a Objetos

O **Polimorfismo** permite que objetos de diferentes classes derivadas sejam tratados como objetos de uma classe base comum, invocando os mesmos métodos, mas executando comportamentos específicos de cada tipo. Isso desacopla o código e permite adicionar novos tipos de classes ao sistema sem a necessidade de alterar a lógica já existente.



O Papel dos Getters e Setters

Os métodos **Getters e Setters** funcionam como uma "camada de proteção" ou "fronteira" em volta dos atributos de uma classe. Em vez de permitir que variáveis de um objeto sejam alteradas diretamente de fora, esses métodos controlam como as informações são lidas e gravadas, garantindo regras de negócio, consistência dos dados e segurança do estado do objeto.

A Função do Getter na Orientação a Objetos

O **Getter** atua como uma "janela de exibição" para o estado de um objeto. Em vez de permitir o acesso direto a uma variável da classe (o que violaria o encapsulamento), o método `get` disponibiliza apenas uma cópia ou leitura controlada do dado, garantindo que outras partes do sistema consigam consultar a informação sem o risco de modificá-la acidentalmente.

```
34 // 4 - Getters
35
36 public Integer getId() {
37     return id;
38 }
39
40 public String getMarca() {
41     return marca;
42 }
43
44 public String getModelo() {
45     return modelo;
46 }
47
48 public int getAno() {
49     return ano;
50 }
```

A Função do Setter na Orientação a Objetos

O **Setter** atua como uma "porta de entrada controlada" para o estado de um objeto. Em vez de permitir que o valor de um atributo privado seja alterado livremente de qualquer lugar do sistema, o método set centraliza a alteração do dado, permitindo validar e proteger a integridade do objeto antes de efetuar a gravação.

```
1 // 5 - Setters
2 public void setId(Integer id) {
3     this.id = id;
4 }
5
6 public void setMarca(String marca) {
7     this.marca = marca;
8 }
9
10 public void setModelo(String modelo)
11     this.modelo = modelo;
12 }
13
14 public void setAno(int ano) {
15     this.ano = ano;
16 }
```



```
Veiculo.java
1 package br.com.gestaofrota.model;
2
3 public class Veiculo {
4
5     // 1 - Atributos
6     private Integer id;
7     private String marca;
8     private String modelo;
9     private int ano;
10
11     // 2 - Construtor
12     public Veiculo() {
13     }
14
15     public Veiculo(Integer id, String marca, String modelo, int ano) {
16         this.id = id;
17         this.marca = marca;
18         this.modelo = modelo;
19         this.ano = ano;
20     }
21
22     public Veiculo(String marca, String modelo, int ano) {
23         this.marca = marca;
24         this.modelo = modelo;
25         this.ano = ano;
26     }
27 }
```

```
27
28 // 3 - Método polimórfico
29 public String exibirFichaTecnica() {
30     return String.format("[%s] %s %s - Ano: %d",
31         getClass().getSimpleName().toUpperCase(), marca, modelo, ano);
32 }
33
34 // 4 - Getters
35 public Integer getId() {
36     return id;
37 }
38
39 public String getMarca() {
40     return marca;
41 }
42
43 public String getModelo() {
44     return modelo;
45 }
46
47 public int getAno() {
48     return ano;
49 }
50
51 // 5 - Setters
52 public void setId(Integer id) {
53     this.id = id;
54 }
55
56 public void setMarca(String marca) {
57     this.marca = marca;
58 }
59
60 public void setModelo(String modelo) {
61     this.modelo = modelo;
62 }
63
64 public void setAno(int ano) {
65     this.ano = ano;
66 }
67 }
68 }
```

O Papel da Subclasse na Herança

Uma **Subclasse** representa uma versão especializada de uma superclasse no paradigma de Orientação a Objetos. Através da relação de herança (IS-A / *É-um*), a subclasse estende as capacidades da classe mãe, reaproveitando sua estrutura de dados e regras de negócio genéricas, ao mesmo tempo em que introduz regras e comportamentos específicos do seu próprio domínio.

A instrução **extends** é o mecanismo fundamental da linguagem Java para promover o reuso de código e a hierarquia de tipos. Ao declarar que uma classe estende outra, o compilador transfere para a nova classe toda a estrutura e comportamentos da classe pai, permitindo ao desenvolvedor focar apenas nas particularidades e regras específicas do novo módulo sem duplicar código.

```
package br.com.gestaofrota.model;

public class Moto extends Veiculo {
    private int cilindradas;

    public Moto(Integer id, String marca, String modelo, int ano, int cilindradas) {
        super(id, marca, modelo, ano);
        this.cilindradas = cilindradas;
    }

    public Moto(String marca, String modelo, int ano, int cilindradas) {
        super(marca, modelo, ano);
        this.cilindradas = cilindradas;
    }

    @Override
    public String exibirFichaTecnica() {
        return super.exibirFichaTecnica() + String.format(" | Cilindradas: %dcc", cilindradas);
    }

    public int getCilindradas() {
        return cilindradas;
    }

    public void setCilindradas(int cilindradas) {
        this.cilindradas = cilindradas;
    }
}
```

A anotação **@Override** é uma diretiva de compilação que valida a sobrescrita de métodos herdados. Além de atuar como uma trava de segurança que impede falhas silenciosas provocadas por erros de digitação ou divergência na assinatura do método, ela melhora significativamente a legibilidade do código, indicando com clareza quais comportamentos da classe mãe foram especializados pela subclasse.

A instrução **extends** é o mecanismo fundamental da linguagem Java para promover o reuso de código e a hierarquia de tipos. Ao declarar que uma classe estende outra, o compilador transfere para a nova classe toda a estrutura e comportamentos da classe pai, permitindo ao desenvolvedor focar apenas nas particularidades e regras específicas do novo módulo sem duplicar código.

```
package br.com.gestaofrota.model;

public class Carro extends Veiculo {
    private int quantidadePortas;

    public Carro(Integer id, String marca, String modelo, int ano, int quantidadePortas) {
        super(id, marca, modelo, ano);
        this.quantidadePortas = quantidadePortas;
    }

    public Carro(String marca, String modelo, int ano, int quantidadePortas) {
        super(marca, modelo, ano);
        this.quantidadePortas = quantidadePortas;
    }

    @Override
    public String exibirFichaTecnica() {
        return super.exibirFichaTecnica() + String.format(" | Portas: %d", quantidadePortas);
    }

    public int getQuantidadePortas() {
        return quantidadePortas;
    }

    public void setQuantidadePortas(int quantidadePortas) {
        this.quantidadePortas = quantidadePortas;
    }
}
```

A anotação **@Override** é uma diretiva de compilação que valida a sobrescrita de métodos herdados. Além de atuar como uma trava de segurança que impede falhas silenciosas provocadas por erros de digitação ou divergência na assinatura do método, ela melhora significativamente a legibilidade do código, indicando com clareza quais comportamentos da classe mãe foram especializados pela subclasse.

Bloco 2:

Exceções Customizadas

Concept Highlight

- Herança de `Exception` para criação de **Checked Exceptions**.
- Separação explícita entre erros técnicos e erros de regra de negócio da aplicação.

O Papel das Exceções no Desenvolvimento Java

As **Exceções** são o mecanismo padrão da linguagem Java para gerenciar condições de erro e imprevistos no sistema. Em vez de permitir que uma falha — como uma consulta a um banco de dados indisponível ou a leitura de um arquivo inexistente — derrube a aplicação inteira, o Java fornece uma estrutura robusta de tratamento que permite capturar a falha, registrar o log e manter a aplicação rodando com segurança.

O Papel das Exceções no Desenvolvimento Java

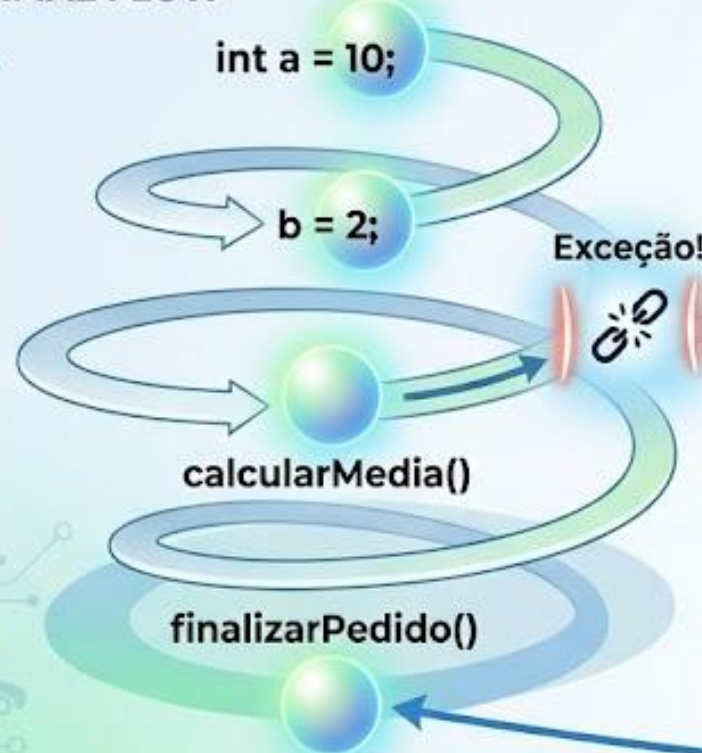
- **O que são:** Eventos anômalos ou erros que ocorrem **durante a execução do programa** e interrompem o fluxo normal das instruções.
- **Objetivo Principal:** Permitir o **tratamento gracioso de erros**, evitando que a aplicação quebre ou seja encerrada de forma abrupta para o usuário.
- **Hierarquia Principal (Throwable):**
 - **Exception:** Situações das quais a aplicação pode se recuperar (ex: arquivo não encontrado, erro de conexão).
 - **Error:** Problemas graves no ambiente da JVM dos quais o programa geralmente não consegue se recuperar (ex: OutOfMemoryError).
- **Tipos de Exceções:**
 - **Checadas (*Checked*):** Verificadas em tempo de compilação; o desenvolvedor é obrigado a tratar ou declarar (ex: IOException).
 - **Não Checadas (*Unchecked / Runtime*):** Erros de lógica ou de programação em tempo de execução (ex: NullPointerException, ArithmeticException).

VISUALIZANDO UMA EXCEÇÃO NO JAVA

INTERRUPÇÃO DO FLUXO NORMAL E TRATAMENTO DE ERROS

NORMAL FLOW

Fluxo Normal de Código



Exceção!



ArithmeticException:
by zero

Stack Trace: br.com...Veiculo.java:45



FLUXO NÃO TRATADO (CRASH)



Aplicação Encerrada
(Erro Crítico)

FLUXO TRATADO (RECUPERAÇÃO)

try-catch
(Recuperação)

finalizarPedido()

Exceção capturada!
e.getMessage():
by zero

logError()

alertarUsuario()

continuarExecucao()

```
try {  
    // Bloco de código propenso a falhas  
    double resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    // Captura e trata o erro específico  
    System.out.println("Erro: Não é possível dividir por zero!");  
} finally {  
    // Bloco opcional que SEMPRE é executado (ex: fechar conexões)  
    System.out.println("Execução finalizada.");  
}
```


O Papel das Exceções Customizadas na Arquitetura

As **Exceções Customizadas** traduzem falhas de regras de negócio em tipos explícitos de erros no sistema. Em vez de lançar exceções genéricas como `IllegalArgumentException` ou `Exception`, a criação de classes específicas de exceção enriquece a camada de domínio, melhora a rastreabilidade nos logs e facilita a captura dessas falhas pelas camadas superiores (como controllers/APIs) para responder mensagens claras ao usuário.

O Papel das Exceções Customizadas na Arquitetura

- **O que são:** Exceções criadas pelo desenvolvedor para representar **erros específicos do negócio** da aplicação (ex: `SaldoInsuficienteException`, `VeiculoNaoEncontradoException`).
- **Por que utilizar?**
 - **Expressividade:** Deixam o código autodocumentado e o motivo do erro claro.
 - **Tratamento Específico:** Permitem capturar e tratar falhas de negócio separadamente de erros técnicos da linguagem.
- **Como criar:**
 - Herda de **Exception** para criar uma exceção *Checada* (*Checked*).
 - Herda de **RuntimeException** para criar uma exceção *Não Checada* (*Unchecked* — mais comum em frameworks modernos).
- **Mecanismo:** Lançada manualmente no código através da palavra-chave **throw new**.

EXCEÇÕES CUSTOMIZADAS NO JAVA (NEGÓCIO)

TRADUZINDO REGRAS DE NEGÓCIO EM CÓDIGO

CLASSE DE EXCEÇÃO PERSONALIZADA

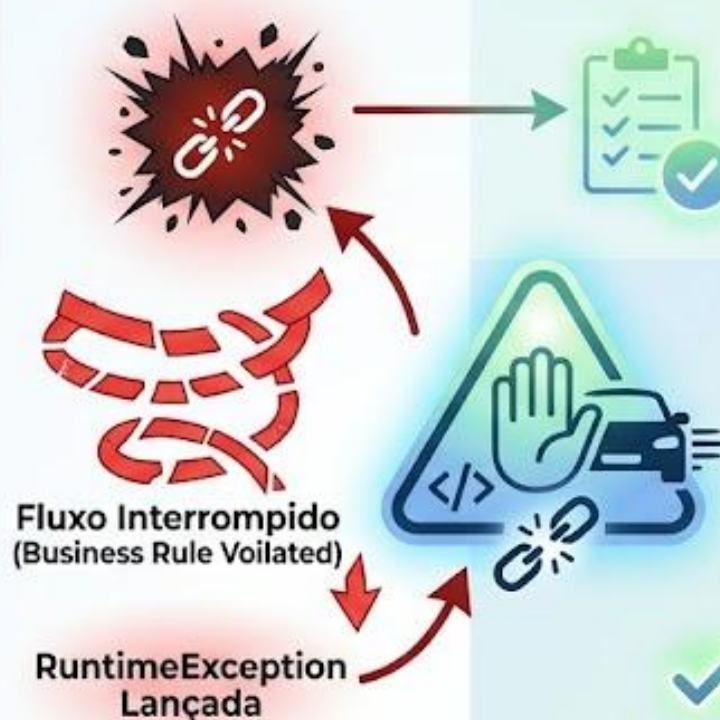
```
public class VeiculoSemMotoristaException  
    extends RuntimeException {  
    public VeiculoSemMotoristaException(String mensagem) {  
        super(mensagem);  
    }  
}
```

USO NA REGRA DE NEGÓCIO

```
if (veiculo.getMotorista() == null) {  
    throw new VeiculoSemMotoristaException(  
        "O veículo " + veiculo.getPlaca() +  
        " não tem motorista associado!");  
}
```

USO NA REGRA DE NEGÓCIO

FLUXO DE ERRO (BLOQUEIO)



FLUXO TRATADO (TRATAMENTO ESPECÍFICO)

Exceção Customizada
Capturada

Exceção
Customizada
Capturada

logError()

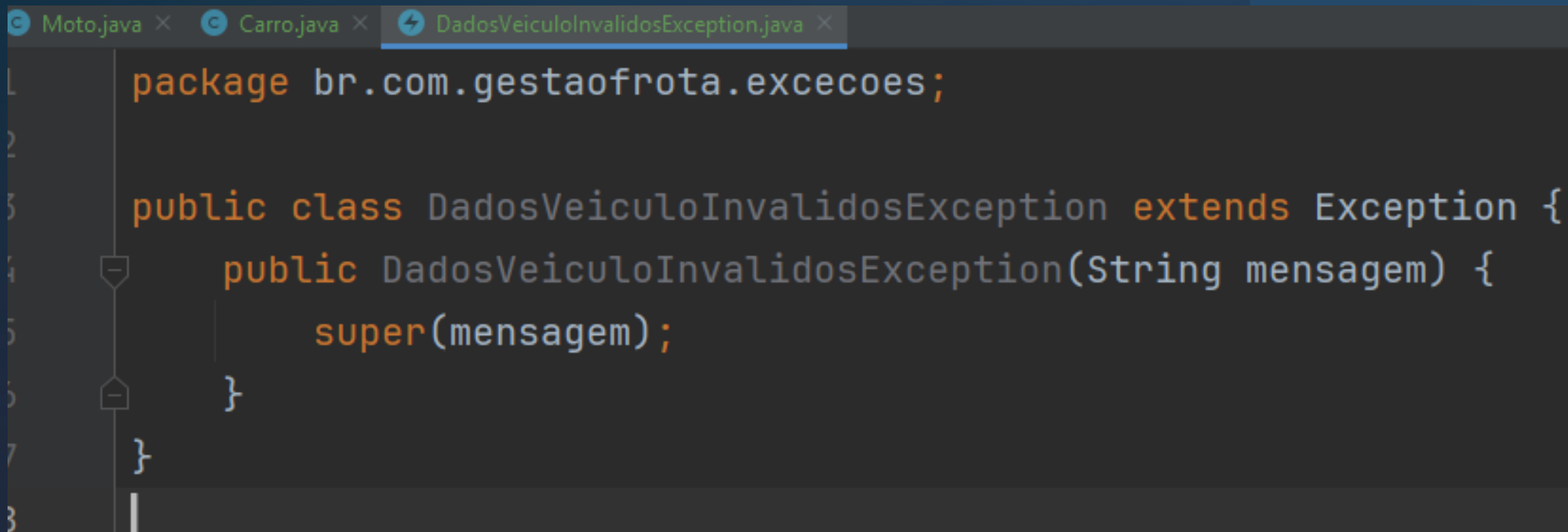
alertarUsuario
("Erro: [mensagem]")

Recuperação Graciosa

```
// 1. Definição da Exceção de Negócio (Herdando de RuntimeException)
public class VeiculoIndisponivelException extends RuntimeException {
    public VeiculoIndisponivelException(String mensagem) {
        super(mensagem); // Repassa a mensagem para a classe pai
    }
}

// 2. Uso na Regra de Negócio (Classe Service)
public void alugarVeiculo(Veiculo veiculo) {
    if (veiculo.isEmManutencao()) {
        // Lançamento manual da exceção customizada
        throw new VeiculoIndisponivelException("O veículo " + veiculo.getPlaca() + " está em manutenção!");
    }
}
```

A instrução **extends** é o mecanismo fundamental da linguagem Java para promover o reuso de código e a hierarquia de tipos. Ao declarar que uma classe estende outra, o compilador transfere para a nova classe toda a estrutura e comportamentos da classe pai, permitindo ao desenvolvedor focar apenas nas particularidades e regras específicas do novo módulo sem duplicar código.

A screenshot of an IDE window with three tabs: 'Moto.java', 'Carro.java', and 'DadosVeiculoInvalidosException.java'. The active tab shows the following Java code:

```
1 package br.com.gestaofrota.excecoes;  
2  
3 public class DadosVeiculoInvalidosException extends Exception {  
4     public DadosVeiculoInvalidosException(String mensagem) {  
5         super(mensagem);  
6     }  
7 }  
8  
9 }
```

Bloco 3:

Integração com Banco de Dados
(JDBC + SQLite)

Concept Highlight

- Conexão simples via **DriverManager**.
- Execução de comandos DDL (CREATE TABLE) e DML (INSERT, SELECT) com PreparedStatement para prevenção de SQL Injection.
- **Manejo de Exceções (SQLException):** Uso de blocos try-with-resources para garantir o fechamento automático das conexões.

O que é a API JDBC (Java Database Connectivity)?

A **JDBC** (*Java Database Connectivity*) é a API padrão da plataforma Java responsável por definir como uma aplicação Java se comunica com sistemas de gerência de banco de dados relacionais (SGBDs), como PostgreSQL, MySQL, SQL Server e Oracle. Ela fornece um conjunto de classes e interfaces que permitem conectar ao banco, executar consultas, atualizar dados e gerenciar transações de forma independente do fabricante do banco.



Principais Componentes da API JDBC

A arquitetura do JDBC é composta por interfaces e classes localizadas principalmente nos pacotes `java.sql` e `javax.sql`:

- **DriverManager**: Gerencia a lista de drivers de banco de dados registrados na aplicação. Ele é responsável por selecionar o driver correto e estabelecer a conexão física com o SGBD.
- **Connection**: Representa a sessão de conexão ativa com o banco de dados. É através dela que controlamos transações (commit, rollback) e criamos comandos SQL.
- **Statement / PreparedStatement**: Interfaces utilizadas para enviar instruções SQL ao banco. O **PreparedStatement** é o mais recomendado por aceitar parâmetros seguros e evitar vulnerabilidades como SQL Injection.
- **ResultSet**: Armazena o resultado retornado por uma consulta (SELECT), permitindo percorrer as linhas e extrair os dados retornados pelo banco.

Como Funciona a Camada de Conexão



API JDBC: A aplicação utiliza chamadas padronizadas do Java.



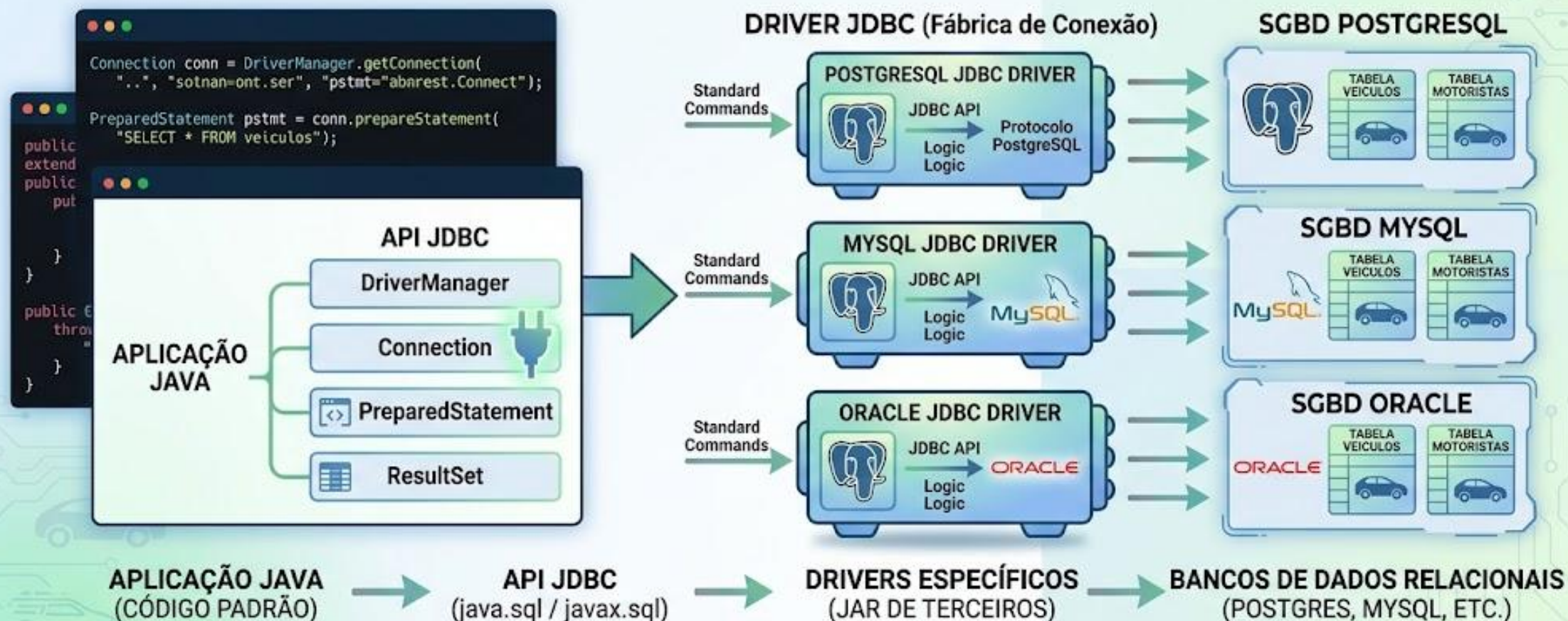
Driver Manager: Localiza o driver adequado com base na URL de conexão informada (ex: jdbc:postgresql://...).



Driver Específico: Cada banco de dados fornece sua própria biblioteca de driver (*JAR*), que implementa as interfaces do JDBC para traduzir os comandos Java em protocolos nativos daquele SGBD.

VISUALIZANDO A API JDBC (JAVA DATABASE CONNECTIVITY)

PADRONIZANDO O ACESSO A DADOS NO JAVA

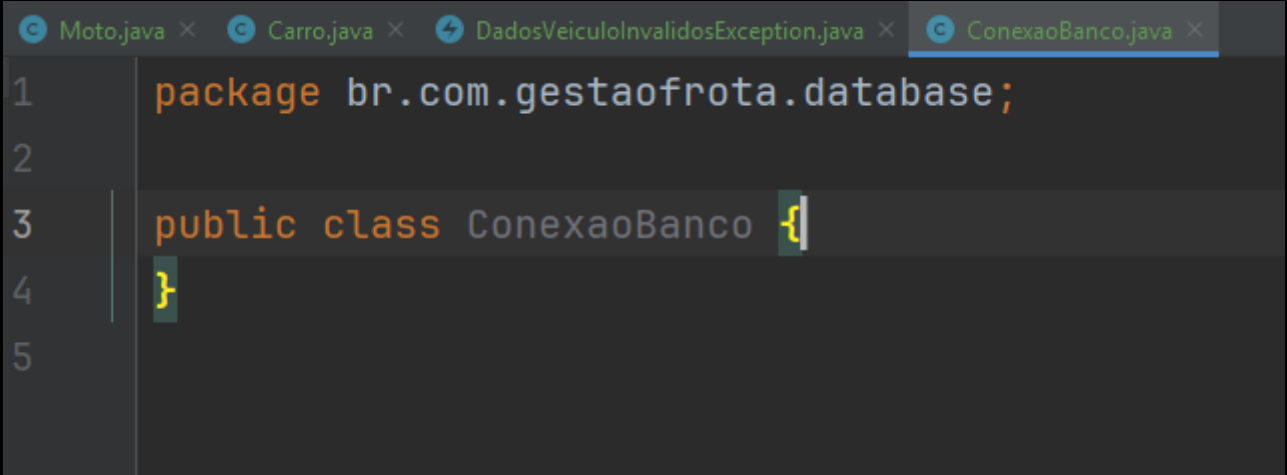


```
String url = "jdbc:postgresql://localhost:5432/gestao_frotas";
String usuario = "admin";
String senha = "123";

// Utiliza o padrão try-with-resources para fechamento automático de conexões
try (Connection conexao = DriverManager.getConnection(url, usuario, senha);
    PreparedStatement stmt = conexao.prepareStatement(
        "SELECT modelo, placa FROM veiculos WHERE ativo = ?")) {

    stmt.setBoolean(1, true);

    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            String modelo = rs.getString("modelo");
            String placa = rs.getString("placa");
            System.out.println("Veículo: " + modelo + " | Placa: " + placa);
        }
    }
} catch (SQLException e) {
    System.out.println("Erro de conexão ou consulta: " + e.getMessage());
}
```



```
Moto.java x Carro.java x DadosVeiculoInvalidosException.java x ConexaoBanco.java x
1 package br.com.gestaofrota.database;
2
3 public class ConexaoBanco {
4 }
5
```

```
Moto.java × MonitorStatusThread.java × ConexaoBanco.java × Main.java ×
1      package br.com.gestaofrota.database;
2
3      import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;
4      import br.com.gestaofrota.model.Carro;
5      import br.com.gestaofrota.model.Moto;
6      import br.com.gestaofrota.model.Veiculo;
7
8      import java.sql.*;
9      import java.util.ArrayList;
10     import java.util.List;
11
12     public class ConexaoBanco {
13
14
15     }
16     |
```

```
package br.com.gestaofrota.database;

import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;
import br.com.gestaofrota.model.Carro;
import br.com.gestaofrota.model.Moto;
import br.com.gestaofrota.model.Veiculo;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ConexaoBanco {

    // Banco de dados em arquivo local 'frota.db' na raiz do projeto
    private static final String URL_CONEXAO = "jdbc:sqlite:frota.db";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(URL_CONEXAO);
    }

    public static void inicializarTabela() {...}

    public static void salvarVeiculo(Veiculo veiculo) throws SQLException, DadosVeiculoInvalidosException {...}

    public static List<Veiculo> listarVeiculos() throws SQLException {...}

}
```



```
public static void inicializarTabela() {  
    String sql = ""  
        CREATE TABLE IF NOT EXISTS veiculos (  
            id INTEGER PRIMARY KEY AUTOINCREMENT,  
            tipo TEXT NOT NULL,  
            marca TEXT NOT NULL,  
            modelo TEXT NOT NULL,  
            ano INTEGER NOT NULL,  
            detalhe_especifico INTEGER NOT NULL  
        );  
    "";  
  
    try (Connection conn = conectar();  
        Statement stmt = conn.createStatement()) {  
        stmt.execute(sql);  
    } catch (SQLException e) {  
        System.err.println("Erro ao inicializar o banco de dados: " + e.getMessage());  
    }  
}
```



```
public static void salvarVeiculo(Veiculo veiculo) throws SQLException, DadosVeiculoInvalidosException {  
    if (veiculo.getAno() < 1900 || veiculo.getAno() > 2027) {  
        throw new DadosVeiculoInvalidosException("Ano do veículo é inválido: " + veiculo.getAno());  
    }  
  
    String sql = "INSERT INTO veiculos(tipo, marca, modelo, ano, detalhe_especifico) VALUES(?, ?, ?, ?, ?)";  
  
    try (Connection conn = conectar();  
        PreparedStatement stmt = conn.prepareStatement(sql)) {  
  
        stmt.setString(parameterIndex: 1, veiculo.getClass().getSimpleName().toUpperCase());  
        stmt.setString(parameterIndex: 2, veiculo.getMarca());  
        stmt.setString(parameterIndex: 3, veiculo.getModelo());  
        stmt.setInt(parameterIndex: 4, veiculo.getAno());  
  
        if (veiculo instanceof Carro c) {  
            stmt.setInt(parameterIndex: 5, c.getQuantidadePortas());  
        } else if (veiculo instanceof Moto m) {  
            stmt.setInt(parameterIndex: 5, m.getCilindradas());  
        }  
  
        stmt.executeUpdate();  
    }  
}
```

```
public static List<Veiculo> listarVeiculos() throws SQLException {  
    List<Veiculo> frota = new ArrayList<>();  
    String sql = "SELECT * FROM veiculos";  
  
    try (Connection conn = conectar();  
        Statement stmt = conn.createStatement();  
        ResultSet rs = stmt.executeQuery(sql)) {  
  
        while (rs.next()) {  
            int id = rs.getInt(columnLabel: "id");  
            String tipo = rs.getString(columnLabel: "tipo");  
            String marca = rs.getString(columnLabel: "marca");  
            String modelo = rs.getString(columnLabel: "modelo");  
            int ano = rs.getInt(columnLabel: "ano");  
            int detalhe = rs.getInt(columnLabel: "detalhe_especifico");  
  
            if ("CARRO".equalsIgnoreCase(tipo)) {  
                frota.add(new Carro(id, marca, modelo, ano, detalhe));  
            } else if ("MOTO".equalsIgnoreCase(tipo)) {  
                frota.add(new Moto(id, marca, modelo, ano, detalhe));  
            }  
        }  
    }  
    return frota;  
}
```

Bloco 4:

Programação Paralela com Threads

Concept Highlight

- Implementação da interface Runnable.
- Execução concorrente sem bloquear a Thread principal (main) que executa o menu de interação com o usuário.
- Controle de execução contínua com Thread.sleep().

O Papel da Programação Paralela em Java

A **programação com Threads** permite que aplicações Java aproveitem ao máximo o poder computacional de processadores *multicore*. Ao dividir operações independentes (como processamento de arquivos, requisições HTTP ou cálculos pesados) em diferentes threads, a aplicação melhora a capacidade de resposta (*responsiveness*) e reduz o tempo total de processamento do sistema.



O Papel da Programação Paralela em Java

- **Thread (Linha de Execução):** É a menor unidade de código executada de forma independente pelo sistema operacional dentro de um processo.
- **Concorrência vs. Paralelismo:**
 - **Concorrência:** Múltiplas tarefas progridem alternando o uso da CPU (*multitasking*).
 - **Paralelismo:** Múltiplas tarefas são executadas **simultaneamente** em núcleos (*cores*) separados do processador.
- **Formas Tradicionais de Criar Threads:**
 - Herdar da classe **Thread** e sobrescrever o método `run()`.
 - Implementar a interface **Runnable** (abordagem mais recomendada e flexível).
- **Desafios e Cuidados:**
 - **Race Conditions (Condições de Corrida):** Quando duas threads tentam alterar o mesmo dado simultaneamente.
 - **Sincronização:** Uso da palavra-chave `synchronized` ou de estruturas atômicas para garantir consistência em ambientes *thread-safe*.

CRIAÇÃO E EXECUÇÃO DE THREADS EM JAVA

COMPREENDENDO O CICLO DE VIDA E O FLUXO DE TRABALHO

MÉTODOS DE CRIAÇÃO

MÉTODO A:

IMPLEMENTANDO Runnable

```
Runnable tarefa = () -> {  
    /* código */  
};  
  
Thread t1 = new Thread(tarefa);
```

MÉTODO B:

HERDANDO DE Thread

```
class MinhaThread extends Thread {  
    @Override  
    public void run() {  
        /* código */  
    }  
}  
  
Thread t2 = new Thread();
```

MÉTODO B:

HERDANDO DE Thread



t.start()

FLUXO DE EXECUÇÃO E ESTADOS

CPU MULTI-CORE (Escalonamento)



FLUXO DE EXECUÇÃO (Estado: RUNNABLE)
EXECUTANDO O MÉTODO run()

Thread-Veiculo-01 Processar Telemetria
Thread-Veiculo-02 Calcular Rota
Thread-Veiculo-03 Calcular Rota

FINALIZADA
(Estado: TERMINATED)

Tarefa concluída

t.wait()



t.join()
t.sleep()

Estados Bloqueados
(Bloqueado/Aguardando/Timed-Waiting)




```
// 1. Definindo a tarefa via interface Runnable
Runnable tarefaManutencao = () -> {
    System.out.println("Processando telemetria na Thread: " + Thread.currentThread().getName());
};

// 2. Instanciando e iniciando as Threads
Thread t1 = new Thread(tarefaManutencao, "Thread-Veiculo-01");
Thread t2 = new Thread(tarefaManutencao, "Thread-Veiculo-02");

t1.start(); // Inicia a execução paralela (chama o método run internamente)
t2.start();
```




```
Moto.java × MonitorStatusThread.java ×  
1 package br.com.gestaofrota.service;  
2  
3 public class MonitorStatusThread {  
4 }  
5
```

```
package br.com.gestaofrota.service;

import br.com.gestaofrota.database.ConexaoBanco;

public class MonitorStatusThread implements Runnable {
    @Override
    public void run() {
        while (true) {
            try {
                int totalVeiculos = ConexaoBanco.listarVeiculos().size();
                System.out.printf("%n[BACKGROUND-THREAD] Sync status: Banco ativo | Total de veículos cadastrados: %d%n> ",
                    totalVeiculos);

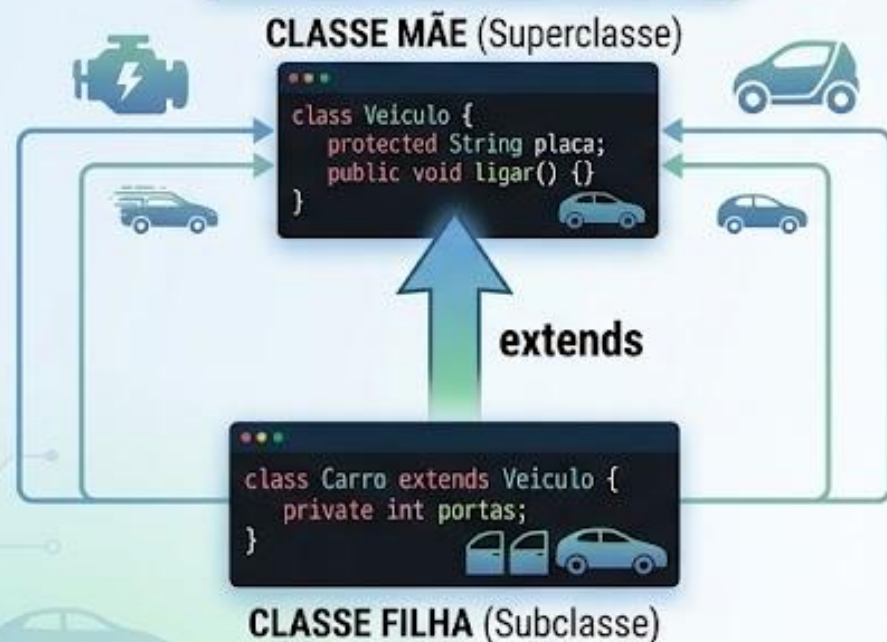
                // Intervalo de verificação: 15 segundos
                Thread.sleep(15000);
            } catch (InterruptedException e) {
                System.out.println("[BACKGROUND-THREAD] Monitor encerrado.");
                break;
            } catch (Exception e) {
                System.err.println("[BACKGROUND-THREAD] Falha de sincronização: " + e.getMessage());
            }
        }
    }
}
```

A instrução **implements** é o mecanismo do Java para garantir o cumprimento de contratos de interface. Ela permite que classes com origens e hierarquias completamente distintas compartilhem um mesmo conjunto de comportamentos e assinaturas de métodos. Isso viabiliza o uso de abstrações, inversão de controle e facilidade de testes unitários na aplicação.

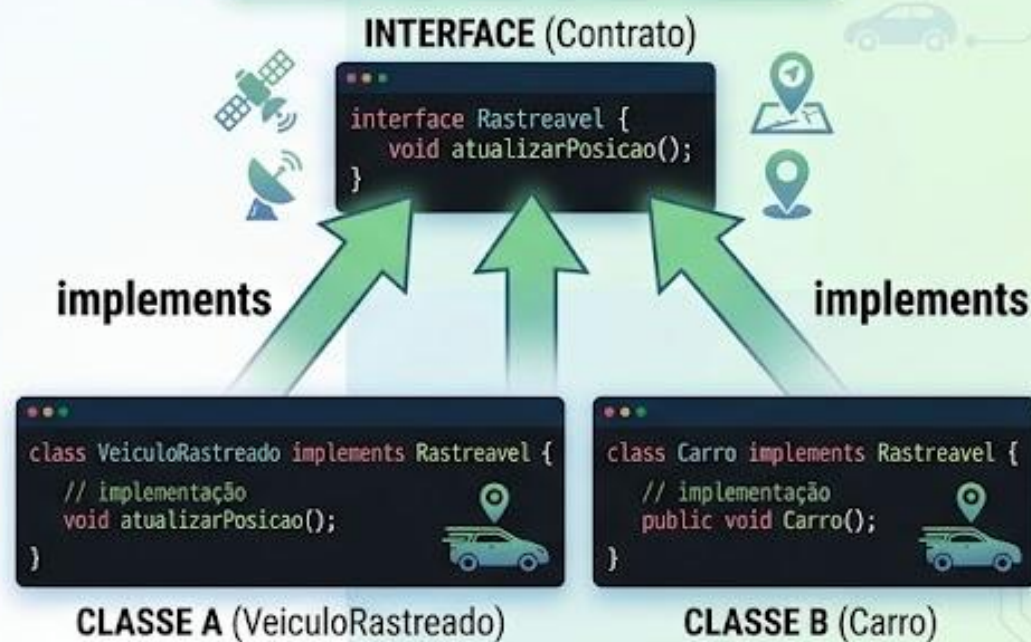
DIFERENÇA ENTRE 'extends' E 'implements' NO JAVA

HERANÇA DE CLASSE vs. CONTRATO DE COMPORTAMENTO

'extends' (HERANÇA)



'implements' (INTERFACE)



Bloco 5:

Classe Principal e Execução

Concept Highlight

- Ponto de entrada do programa.
- Inicialização da Thread concorrente via `new Thread(runnable).start()`.
- Menu interativo via Scanner exercitando a adição de carros/motos e a listagem polimórfica.

O Papel da Classe Principal na Arquitetura

A Classe Principal funciona como o "orquestrador inicial" da aplicação. Ela é o primeiro código carregado e executado pela Java Virtual Machine (JVM). Sua função primária é configurar o ambiente, inicializar dependências, instanciar as classes do sistema e invocar o fluxo de inicialização da regra de negócio ou da interface de usuário.

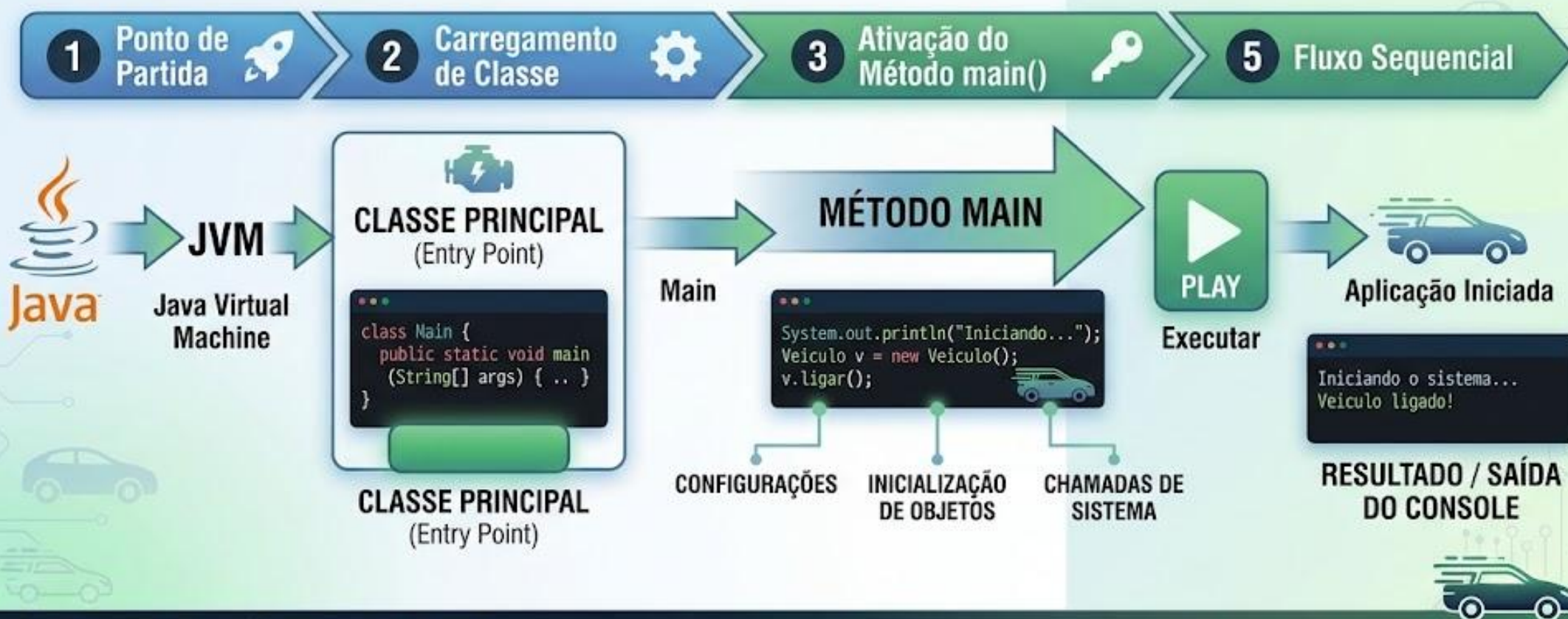
O Papel da Classe Principal na Arquitetura

- **O que é:** É a classe utilizada como **ponto de entrada (*entry point*)** da aplicação Java para dar início à sua execução.
- **O Método main:** Para que uma classe seja executada pela máquina virtual (JVM), ela precisa conter obrigatoriamente a assinatura exata: `public static void main(String[] args)`
- **Anatomia do Método main:**
 - **public:** Visível e acessível pela JVM de qualquer lugar.
 - **static:** Permite que a JVM execute o método sem precisar instanciar um objeto da classe.
 - **void:** Não retorna nenhum valor após finalizar.
 - **String[] args:** Vetor que recebe parâmetros/argumentos passados via linha de comando.
- **Fluxo de Execução:** A JVM carrega o bytecode (.class), localiza o método main e executa suas instruções sequencialmente.

CLASSE PRINCIPAL E FLUXO DE EXECUÇÃO

COMPREENDENDO O PONTO DE ENTRADA DO SISTEMA

PROCESSO DE INICIALIZAÇÃO



01

```
Moto.java x MonitorStatusThread.java x ConexaoBanco.java x Main.java x
1 package br.com.gestaofrota;
2
3 import br.com.gestaofrota.database.ConexaoBanco;
4 import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;
5 import br.com.gestaofrota.model.Carro;
6 import br.com.gestaofrota.model.Moto;
7 import br.com.gestaofrota.model.Veiculo;
8 import br.com.gestaofrota.service.MonitorStatusThread;
9
10 import java.sql.SQLException;
11 import java.util.List;
12 import java.util.Scanner;
13
14 public class Main {
15
16     public static void main(String[] args) {
17
18     }
19 }
```

```
16 ▶ public static void main(String[] args) {  
17     // 1. Inicializa o schema no SQLite  
18     ConexaoBanco.inicializarTabela();  
19  
20     // 2. Inicia a Thread de Monitoramento em Segundo Plano  
21     Thread monitorThread = new Thread(new MonitorStatusThread());  
22     monitorThread.setDaemon(true); // Permite finalizar a JVM sem travar na thread  
23     monitorThread.start();  
24  
25     Scanner scanner = new Scanner(System.in);  
26     int opcao = -1;  
27  
28     System.out.println("=====");  
29     System.out.println("  SISTEMA DE GESTÃO DE FROTA - AULA 01");  
30     System.out.println("=====");  
31  
32 }  
33 }  
34
```

```
while (opcao != 0) {  
    System.out.println("\n--- MENU PRINCIPAL ---");  
    System.out.println("1. Cadastrar Carro");  
    System.out.println("2. Cadastrar Moto");  
    System.out.println("3. Listar Frota Completa (Polimorfismo)");  
    System.out.println("0. Sair");  
    System.out.print("> Escolha uma opção: ");  
  
    try {  
        opcao = Integer.parseInt(scanner.nextLine());  
  
        switch (opcao) {  
            case 1 → cadastrarCarro(scanner);  
            case 2 → cadastrarMoto(scanner);  
            case 3 → listarFrota();  
            case 0 → System.out.println("Encerrando a aplicação...");  
            default → System.out.println("Opção inválida!");  
        }  
    } catch (NumberFormatException e) {  
        System.out.println("Erro: Digite apenas números inteiros para o menu.");  
    }  
}  
scanner.close();
```

04

```
Moto.java x MonitorStatusThread.java x ConexaoBanco.java x Main.java x
1      package br.com.gestaofrota;
2
3      import br.com.gestaofrota.database.ConexaoBanco;
4      import br.com.gestaofrota.excecoes.DadosVeiculoInvalidosException;
5      import br.com.gestaofrota.model.Carro;
6      import br.com.gestaofrota.model.Moto;
7      import br.com.gestaofrota.model.Veiculo;
8      import br.com.gestaofrota.service.MonitorStatusThread;
9
10     import java.sql.SQLException;
11     import java.util.List;
12     import java.util.Scanner;
13
14     public class Main {
15
16     public static void main(String[] args) {...}
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57     @ private static void cadastrarCarro(Scanner scanner) {...}
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81     @ private static void cadastrarMoto(Scanner scanner) {...}
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105     private static void listarFrota() {...}
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124 }
125
```

```
private static void cadastrarCarro(Scanner scanner) {  
    try {  
        System.out.print("Marca: ");  
        String marca = scanner.nextLine();  
        System.out.print("Modelo: ");  
        String modelo = scanner.nextLine();  
        System.out.print("Ano: ");  
        int ano = Integer.parseInt(scanner.nextLine());  
        System.out.print("Quantidade de Portas: ");  
        int portas = Integer.parseInt(scanner.nextLine());  
  
        Carro carro = new Carro(marca, modelo, ano, portas);  
        ConexaoBanco.salvarVeiculo(carro);  
        System.out.println("✅ Carro cadastrado e salvo no banco com sucesso!");  
  
    } catch (DadosVeiculoInvalidosException e) {  
        System.err.println("Regra de Negócio: " + e.getMessage());  
    } catch (SQLException e) {  
        System.err.println("Erro de Banco de Dados: " + e.getMessage());  
    } catch (NumberFormatException e) {  
        System.err.println("Entrada inválida: Certifique-se de digitar números para Ano/Portas.");  
    }  
}
```

```
private static void cadastrarMoto(Scanner scanner) {
    try {
        System.out.print("Marca: ");
        String marca = scanner.nextLine();
        System.out.print("Modelo: ");
        String modelo = scanner.nextLine();
        System.out.print("Ano: ");
        int ano = Integer.parseInt(scanner.nextLine());
        System.out.print("Cilindradas: ");
        int cc = Integer.parseInt(scanner.nextLine());

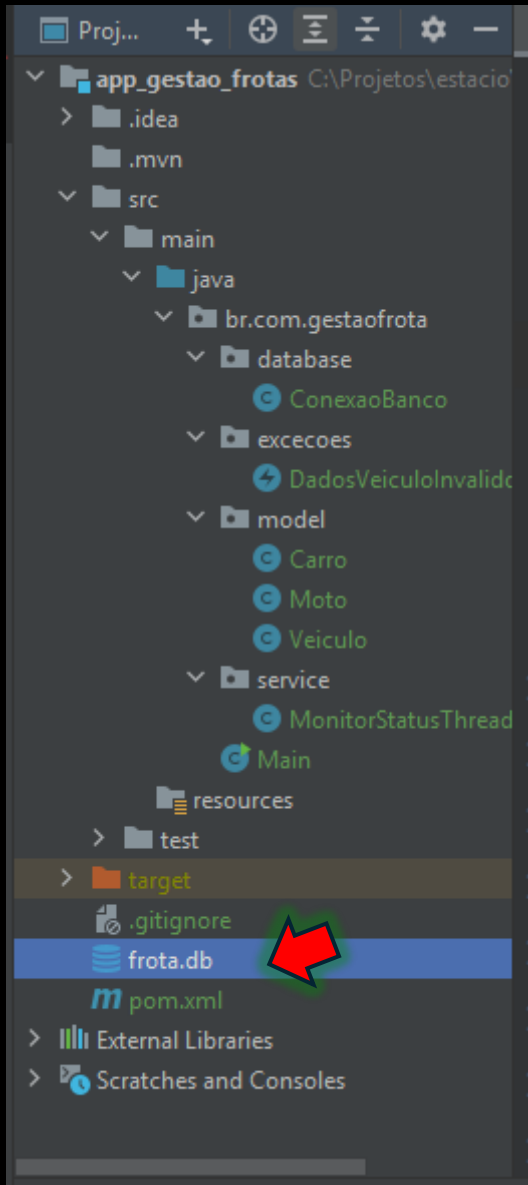
        Moto moto = new Moto(marca, modelo, ano, cc);
        ConexaoBanco.salvarVeiculo(moto);
        System.out.println("✅ Moto cadastrada e salva no banco com sucesso!");

    } catch (DadosVeiculoInvalidosException e) {
        System.err.println("Regra de Negócio: " + e.getMessage());
    } catch (SQLException e) {
        System.err.println("Erro de Banco de Dados: " + e.getMessage());
    } catch (NumberFormatException e) {
        System.err.println("Entrada inválida: Certifique-se de digitar números para Ano/Cilindradas.");
    }
}
```

```
private static void listarFrota() {  
    try {  
        List<Veiculo> frota = ConexaoBanco.listarVeiculos();  
        System.out.println("\n≡≡≡ FROTA CADASTRADA ≡≡≡");  
  
        if (frota.isEmpty()) {  
            System.out.println("Nenhum veículo encontrado no banco de dados.");  
            return;  
        }  
  
        // Exibição polimórfica  
        for (Veiculo v : frota) {  
            System.out.println(v.exibirFichaTecnica());  
        }  
  
    } catch (SQLException e) {  
        System.err.println("Erro ao consultar a frota: " + e.getMessage());  
    }  
}
```


A low-angle, close-up shot of a person's legs and feet while running on a track. The runner is wearing dark-colored athletic shoes with a prominent white sole and a textured tread pattern. The background is heavily blurred with horizontal streaks of yellow, orange, and blue, suggesting high speed and a bright, sunny environment. The overall tone is energetic and dynamic.

Executando a aplicação



```
main
C:\Users\fabricaoleonard\.jdk\azul-21.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA\bin\idea_rt.jar=53366"
-Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Projetos\estacio\app_gestao_frotas\target\classes;
C:\Users\fabricaoleonard\.m2\repository\org\xerial\sqlite-jdbc\3.45.1.0\sqlite-jdbc-3.45.1.0.jar;
C:\Users\fabricaoleonard\.m2\repository\org\slf4j\slf4j-api\1.7.36\slf4j-api-1.7.36.jar br.com.gestaofrota.Main

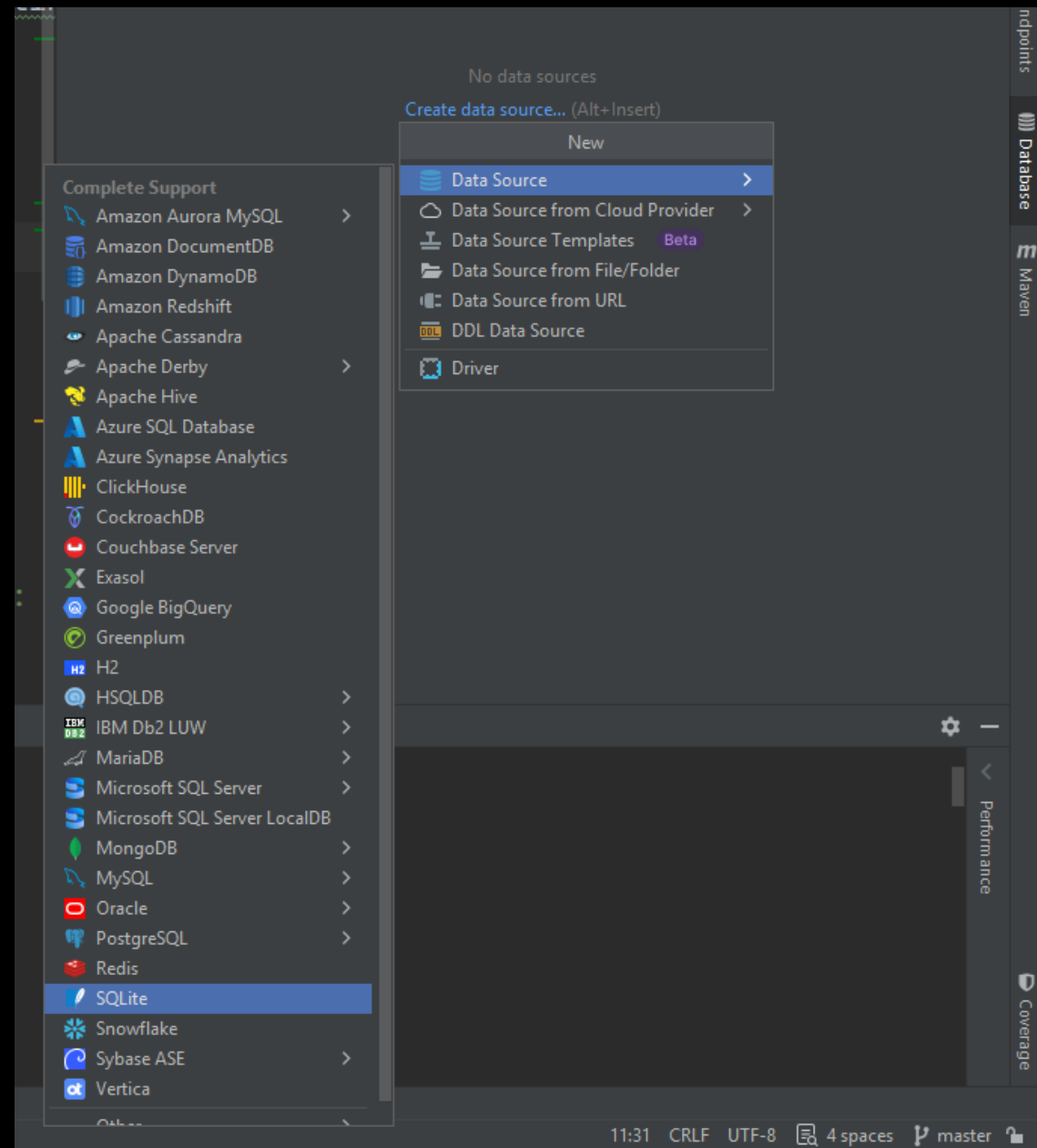
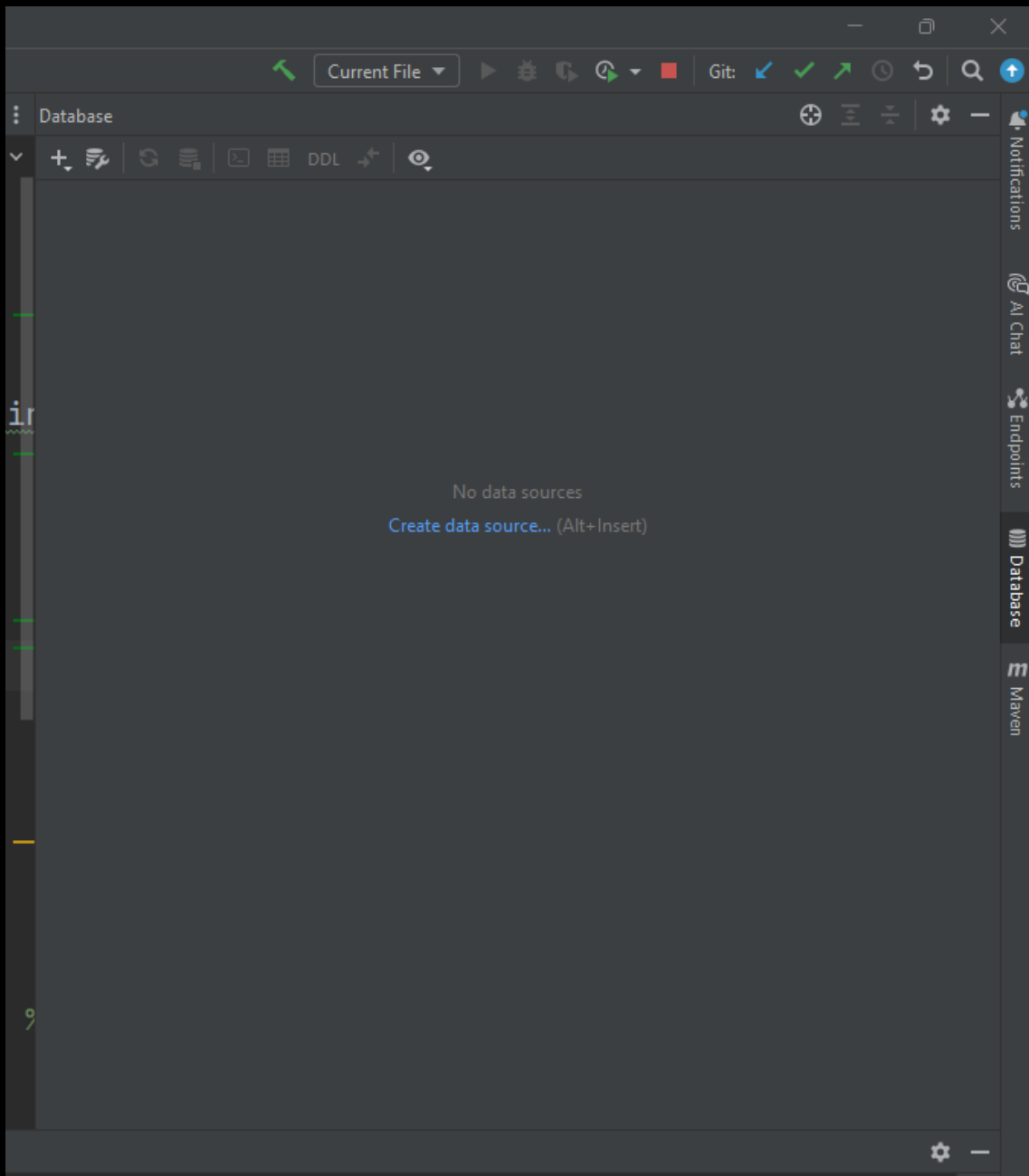
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.

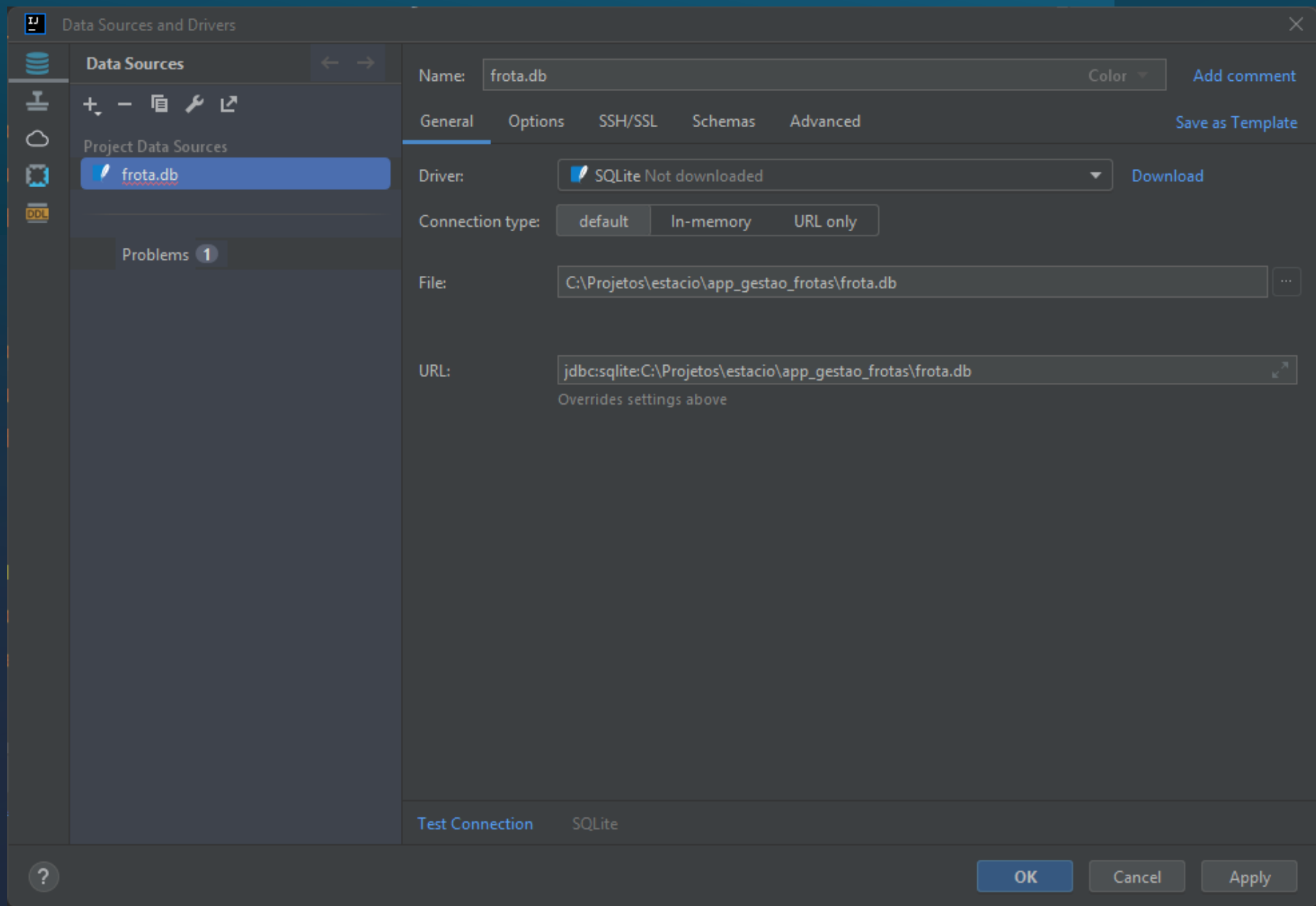
[BACKGROUND-THREAD] Sync status: Banco ativo | Total de veículos cadastrados: 0
> ===+=====
    SISTEMA DE GESTÃO DE FROTA - AULA 01
=====

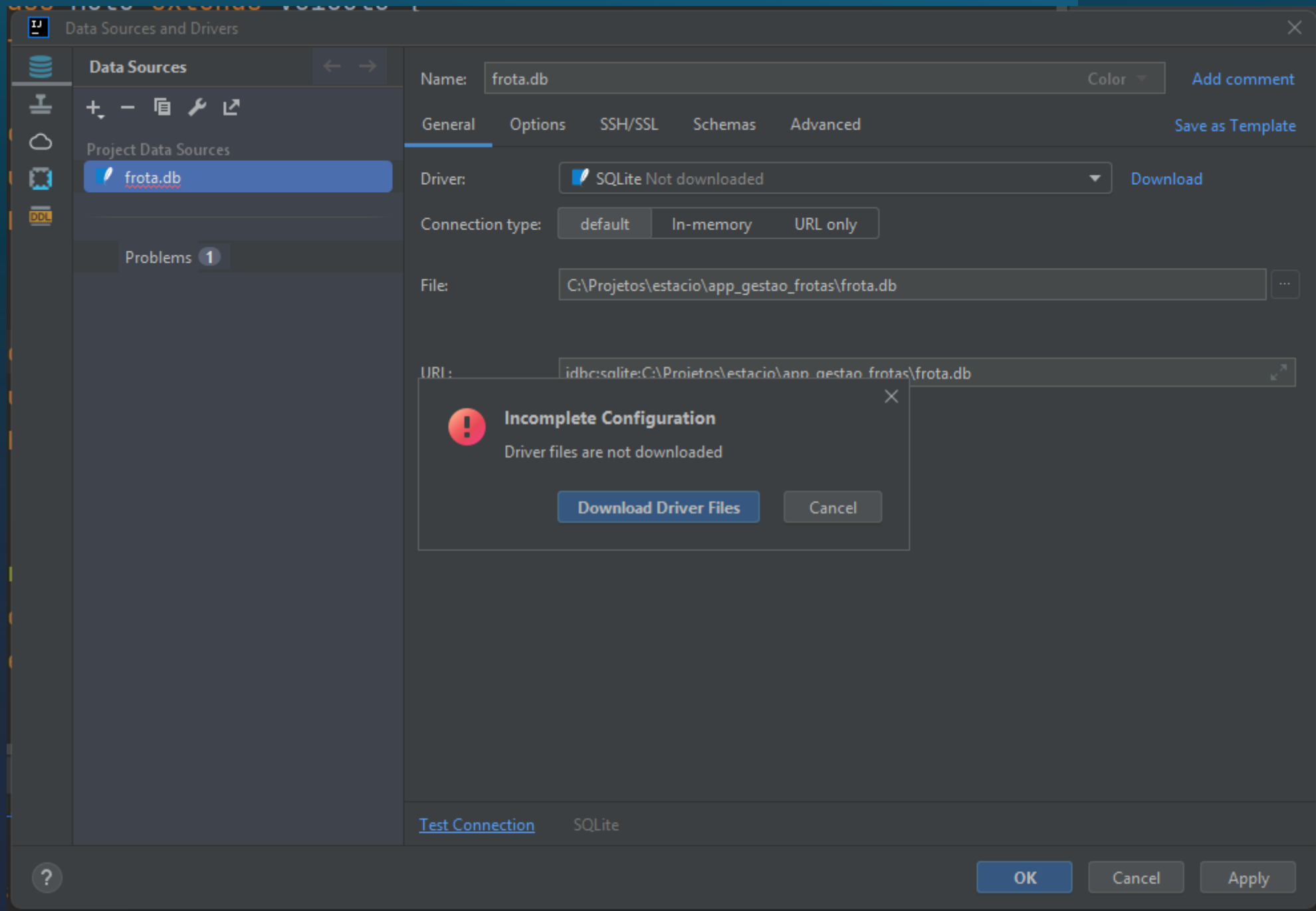
--- MENU PRINCIPAL ---
1. Cadastrar Carro
2. Cadastrar Moto
3. Listar Frota Completa (Polimorfismo)
0. Sair
> Escolha uma opção: 1
Marca: Fiat
Modelo: 2023
Ano: 2023
Quantidade de Portas: 4
[BACKGROUND-THREAD] Sync status: Banco ativo | Total de veículos cadastrados: 0
>
✓ Carro cadastrado e salvo no banco com sucesso!
```


The image shows a perspective view of a row of computer workstations. Each workstation consists of a light-colored wooden desk, a blue ergonomic office chair, and a black computer monitor. The monitors have a distinctive design with a large, curved, perforated area on the back, likely for ventilation or as a design element. The workstations are arranged in a line, receding into the background. The text "Banco de dados" is overlaid in the center of the image.

Banco de dados









Data Sources and Drivers



Data Sources



Project Data Sources



frota.db



Problems

Name: frota.db

Color ▾

Add comment

General

Options

SSH/SSL

Schemas

Advanced

Save as Template

Driver:

SQLite

Connection type:

default

In-memory

URL only

File:

C:\Projetos\estacio\app_gestao_frotas\frota.db

URL:

jdbc:sqlite:C:\Projetos\estacio\app_gestao_frotas\frota.db

Overrides settings above

Succeeded

Copy

DBMS: SQLite (ver. 3.51.1)

Case sensitivity: plain=mixed, delimited=mixed

Driver: SQLite JDBC (ver. 3.51.1.0, JDBC4.2)

Ping: 29 ms

Test Connection



SQLite



OK

Cancel

Apply

File Edit View Navigate Code Refactor Build Run Tools Git Window Help

app_gestao_frotas - pom.xml (app_gestao_frotas)

Database Consoles frota.db console [frota.db]

Project

app_gestao_frotas C:\Projetos\estacio

main

java

br.com.gestaofrota

database

ConexaoBanco

excecoes

DadosVeiculoInvalido

model

Carro

Moto

Veiculo

service

MonitorStatusThread

Main

resources

test

target

.gitignore

frota.db

pom.xml

External Libraries

Scratches and Consoles

Moto.java console [frota.db] MonitorStatusThread.java ConexaoBanco.java Main.java

1 SELECT 998 ms

2 *

3 FROM

4 veiculos

Database

frota.db 1

main

tables 3

sqlite_master

sqlite_sequence

veiculos

columns 6

id INTEGER (auto increment)

tipo TEXT

marca TEXT

modelo TEXT

ano INTEGER

detalhe_especifico INTEGER

keys 1

Server Objects

Services

Docker

Dev Containers

Containers

Images

Networks

Volumes

Database

frota.db

console 998 ms

Output main.veiculos

id tipo marca mode... ano detal...

1 1 CARRO Fiat 2023 2023 4

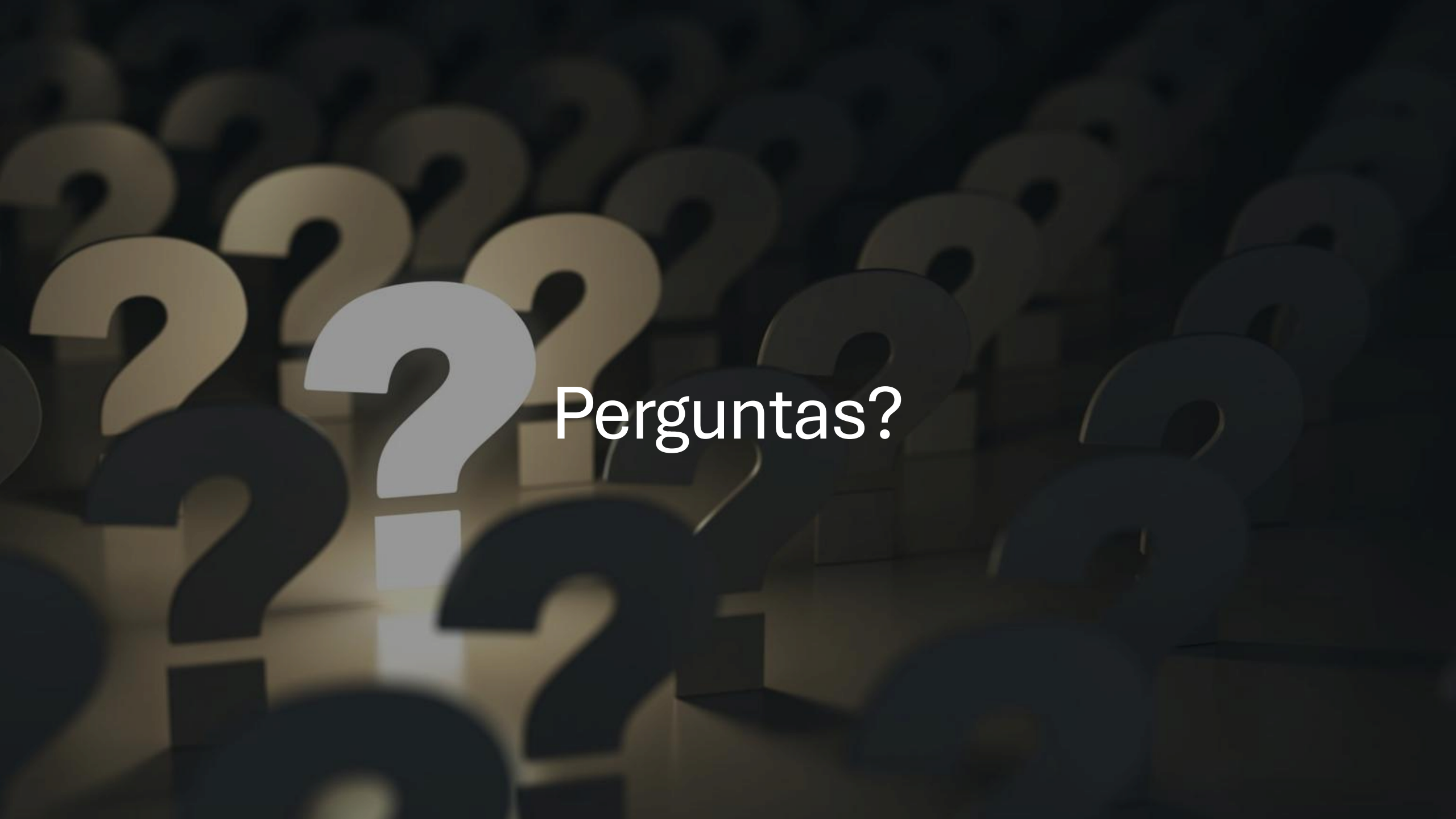
2 2 MOTO Honda Honda 2025 100

2 rows

Git Run Problems Terminal TODO Profiler Services Build

2 rows retrieved starting from 1 in 919 ms (execution: 15 ms, fetching: 904 ms)

4:9 CRLF UTF-8 4 spaces master



Perguntas?